Mining Data Streams (Part 2)

(1) Filtering Data Streams

Filtering Data Streams

- **Each element of data stream is a tuple**
- Given a list of keys **S**
- **Determine which tuples of stream are in *S***

- **Obvious solution: Hash table**
  - But suppose we **do not have enough memory** to store all of ***S*** in a hash table
    - E.g., we might be processing millions of filters on the same stream

Filtering Data Streams

- Each element of data stream is a tuple.
- Given a list of keys S.
- Determine which tuples of stream are in S.
- Obvious solution: Hash table
  - But suppose we do not have enough memory to store all of S in a hash table.
    - E.g., we might be processing millions of filters on the same stream.

Example:

Stream of user ids → [101] [205] [310] [101] [450] [205] [700] ...

Set $S = \{101, 205, 450\}$

For each incoming tuple, check whether its key $\in S$.

Why hash table?



Problem:

If S is very large (e.g., 100 million keys), storing all keys in memory may be expensive.

Also, we may need to maintain millions of filters on the same stream.

Our goal:

Check membership using much less memory.

Applications

- **Example: Email spam filtering**
  - We know 1 billion "good" email addresses
  - If an email comes from one of these, it is **NOT** spam
- **Publish-subscribe systems**
  - You are collecting lots of messages (news articles)
  - People express interest in certain sets of keywords
  - Determine whether each message matches user's interest

Applications

(1) Example: Email spam filtering

- We know 1 billion "good" email addresses.
- If an email comes from one of these, it is **NOT** spam.

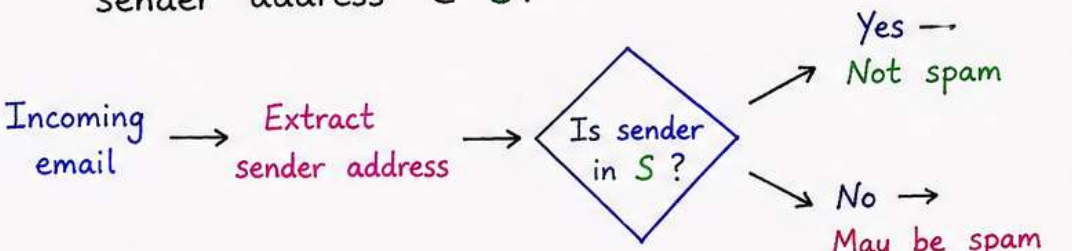
Example:

- abc@university.edu → in S → Not spam
- unknown@xyz.com → not in S → May be spam (needs further checks)

Note: Storing 1 billion addresses in a hash table needs a lot of memory. Bloom Filter helps here.

Idea:

Maintain a set S of good email addresses. For each incoming email, check if the sender address $\in S$.



(2) Publish-subscribe systems

- You are collecting lots of messages (news articles).
- People express interest in certain sets of keywords.
- Determine whether each message matches user's interest.

Example:

User A: { AI, Machine Learning, NLP }

User B: { Cricket, IPL }

User C: { Stock Market, Finance }

⋮

Working:

A news article (message) arrives. We quickly check if it contains any keyword that a user is interested in.

Example message:

"New AI model improves machine translation."

Match users:

Matches → User A (because of keywords: AI)

Not match → User B, User C

Main idea in both applications:

For every incoming item in the stream, quickly decide whether it belongs to a given set S.

First Cut Solution (1)

- **Given a set of keys *S* that we want to filter**
- Create a **bit array *B*** of ***n*** bits, initially all ***0*s**
- Choose a **hash function *h*** with range **[*0,n*)**
- Hash each member of $s \in S$ to one of ***n*** buckets, and set that bit to **1**, i.e., ***B[h(s)]=1***
- Hash each element ***a*** of the stream and output only those that hash to bit that was set to **1**
  - **Output *a* if B[h(a)] == 1**

First Cut Solution (1)

- Given a set of keys S that we want to filter.
- Create a bit array B of n bits, initially all 0s.
- Choose a hash function h with range $[0, n)$.
- Hash each member of $s \in S$ to one of n buckets, and set that bit to 1, i.e., $B[h(s)] = 1$.
- Hash each element a of the stream and output only those that hash to bit that was set to 1.
  - Output a if $B[h(a)] == 1$.

Example

Let $S = \{25, 36, 42\}$

$n = 10 \Rightarrow$ bit array B has 10 bits

Initially,

$B =$

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

Choose hash function $h(x) = x \bmod 10$

Hash each member of S:

$h(25) = 5 \Rightarrow$ set $B[5] = 1$

$h(36) = 6 \Rightarrow$ set $B[6] = 1$

$h(42) = 2 \Rightarrow$ set $B[2] = 1$

After inserting all members of S,

$B =$

0	1	2	3	4	5	6	7	8	9
0	0	1	0	0	1	1	0	0	0

Querying (processing stream elements)

- Suppose $a = 36$

  $h(36) = 36 \bmod 10 = 6$

  $B[6] = 1 \Rightarrow$ Output 36 (may be in S)

- Suppose $a = 47$

  $h(47) = 47 \bmod 10 = 7$

  $B[7] = 0 \Rightarrow$ Do NOT output 47 (definitely not in S)

False Positive (possible)

- Suppose $a = 15$

  $h(15) = 15 \bmod 10 = 5$

  $B[5] = 1$ (already set by 25)

  $\Rightarrow$ Output 15

  (But 15 is NOT in S)

Important Observation

- If $B[h(a)] = 0$

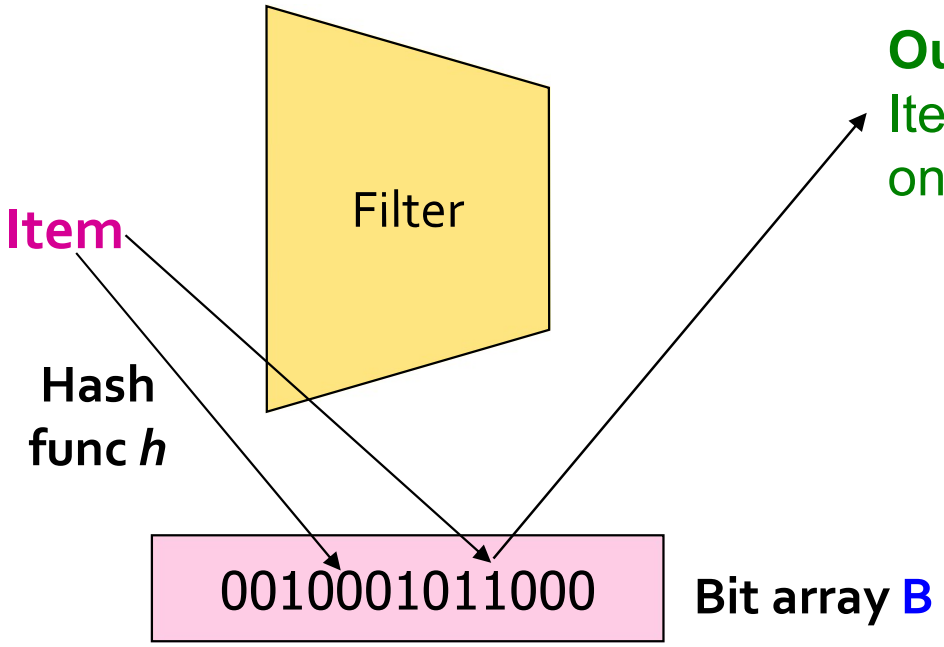
  $\Rightarrow a$ is definitely NOT in S.

- If $B[h(a)] = 1$

  $\Rightarrow a$ may be in S

  (there can be false positives).

First Cut Solution (2)

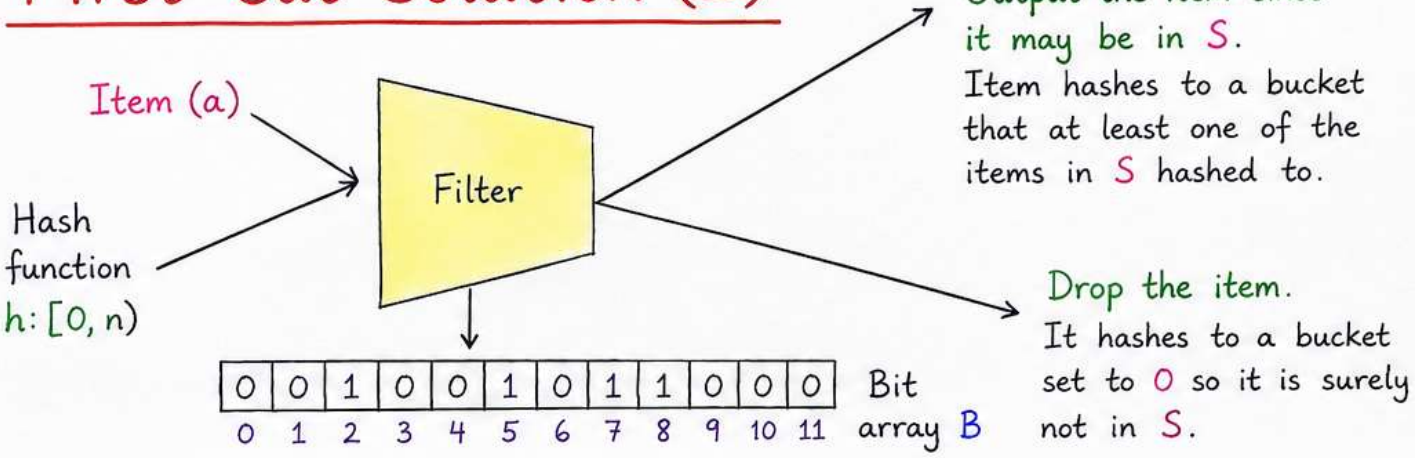


Output the item since it may be in *S*.
Item hashes to a bucket that at least one of the items in **S** hashed to.

Drop the item.
It hashes to a bucket set to **0** so it is surely not in ***S***.

- **Creates false positives but no false negatives**
  - If the item is in ***S*** we surely output it, if not we may still output it

First Cut Solution (2)



Output the item since it may be in S.
Item hashes to a bucket that at least one of the items in S hashed to.

Drop the item.
It hashes to a bucket set to 0 so it is surely not in S.

- Creates false positives but no false negatives

Case 1: Bit = 0 → Drop

Suppose $h(a) = 6$ and $B[6] = 0$

Then a is definitely NOT in S.

Why? If a had been in S earlier, we would have set $B[h(a)] = 1$.

So, $B[h(a)] = 0 \Rightarrow a \notin S$

Therefore, DROP the item.

Case 2: Bit = 1 → Output

Suppose $h(a) = 7$ and $B[7] = 1$

Now, we cannot say that a is definitely in S.

Because some other item may have hashed to the same position.

Example:

$S = \{25\}$, $h(x) = x \bmod 10$

$h(25) = 5 \Rightarrow B[5] = 1$

Later, item 15 arrives.

$h(15) = 5$ and $B[5] = 1$

Filter outputs 15 even though $15 \notin S$.

This is a FALSE POSITIVE.

Why NO false negatives?

Suppose $a \in S$.

When a was inserted, we definitely set $B[h(a)] = 1$.

Therefore, when a arrives later, it will see $B[h(a)] = 1$ and will be output.

So, $a \in S \Rightarrow$ filter will output a.

Key takeaway

- Bit = 0 ⇒ Definitely NOT present.
- Bit = 1 ⇒ Possibly present.

In this first-cut solution (one hash function), the filter may give FALSE POSITIVES but never FALSE NEGATIVES.

First Cut Solution (3)

- **|S| = 1 billion email addresses**
  |B|= 1GB = 8 billion bits
- If the email address is in ***S***, then it surely hashes to a bucket that has the big set to **1**, so it always gets through (***no false negatives***)
- Approximately **1/8** of the bits are set to **1**, so about **1/8th** of the addresses not in ***S*** get through to the output (***false positives***)
  - Actually, less than **1/8th**, because more than one address might hash to the same bit

First Cut Solution (3)

- Given: $|S|$ = 1 billion email addresses
  $|B|$ = 1 GB = 8 billion bits

  Idea: Instead of storing all addresses, use a bit array B of 8 billion bits. We hash each address to one bit.

- Number of 1's in B:

  We insert 1 billion addresses into 8 billion bit positions.

  If there were no collisions,

  fraction of 1's ≈ 1 billion / 8 billion = 1/8

  So approximately 1/8 of the bits are set to 1.

Example (conceptual):

B has 16 bits (small example)

0	1	0	0	1	0	0	1	0	0	1	0	0	0	1	0
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

index:

Why no false negatives?

If an email address $a \in S$, when we inserted it we did:

$h(a) = j \Rightarrow B[j] = 1$

Therefore, when a arrives again,

$B[h(a)] = 1 \Rightarrow$ it will always pass.

So there are NO FALSE NEGATIVES.

- Where do false positives come from?

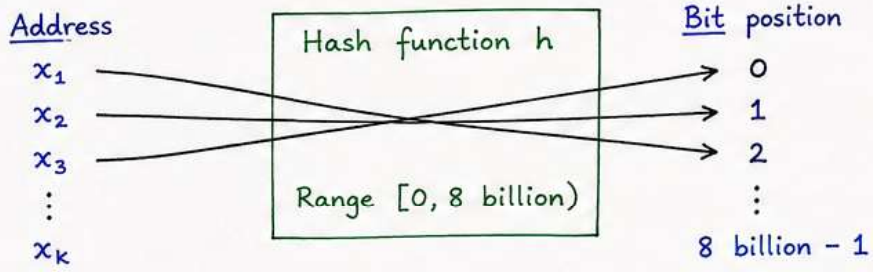
  An address a that is NOT in S might hash to a bit that is already 1.

  $a \notin S$ but $B[h(a)] = 1 \Rightarrow$ a is output

  This is a FALSE POSITIVE.

Why does this happen?

Because many different addresses can hash to the same bit.



- Probability of false positive

  If about 1/8 of the bits are 1, then a random address not in S has about 1/8 chance of landing on a 1.

  P(false positive) ≈ 1/8 = 12.5%

  - Actually less than 1/8, because more than one address may hash to the same bit (collisions).

  P(false positive) < 1/8

Key takeaway:

- Bit = 0 => Definitely NOT in S (drop the item)
- Bit = 1 => Possibly in S (output the item)

First-cut solution: false positives possible, but NO false negatives.

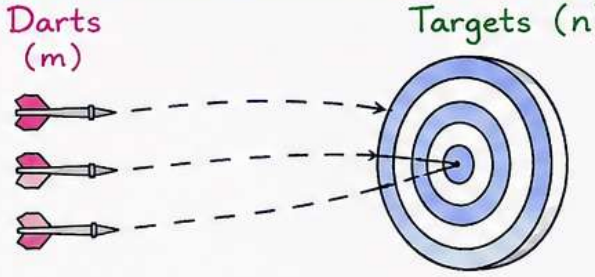
Analysis: Throwing Darts (1)

- **More accurate analysis for the number of false positives**
- **Consider:** If we throw ***m*** darts into ***n*** equally likely targets, **what is the probability that a target gets at least one dart?**
- **In our case:**
  - **Targets** = bits/buckets
  - **Darts** = hash values of items

Analysis: Throwing Darts (1)

- More accurate analysis for the number of false positives.
- Consider: If we throw m darts into n equally likely targets, what is the probability that a target gets at least one dart?
- In our case:
  - Targets = bits/buckets in the bit array B
  - Darts = hash values of items in S

DARTS – TARGETS ANALOGY



We want:
Probability that a particular target gets at least one dart.

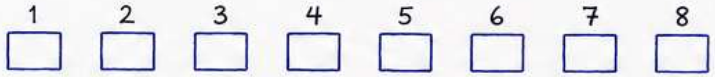
Why this analogy?

When we insert items of S, each item is hashed to one of the n bit positions uniformly at random.
This is exactly like throwing darts at n targets.

A bit becomes 1 if at least one item hashes to it.
So, probability (false positive) = probability (a bit is 1)
= probability (a target gets at least one dart)

Simple example:

Suppose $n = 8$ targets (bits), $m = 3$ darts (items)



Throw 3 darts:
Item A → target 2
Item B → target 5
Item C → target 2

A and C hit the same target.
So only 2 targets get at least one dart.

Resulting bit array B:



Key idea:

- If there were no collisions, then $\frac{m}{n}$ of the bits would be 1.
- But collisions happen (multiple items may hash to the same bit), so the fraction of 1's is less than $\frac{m}{n}$.

How to compute?

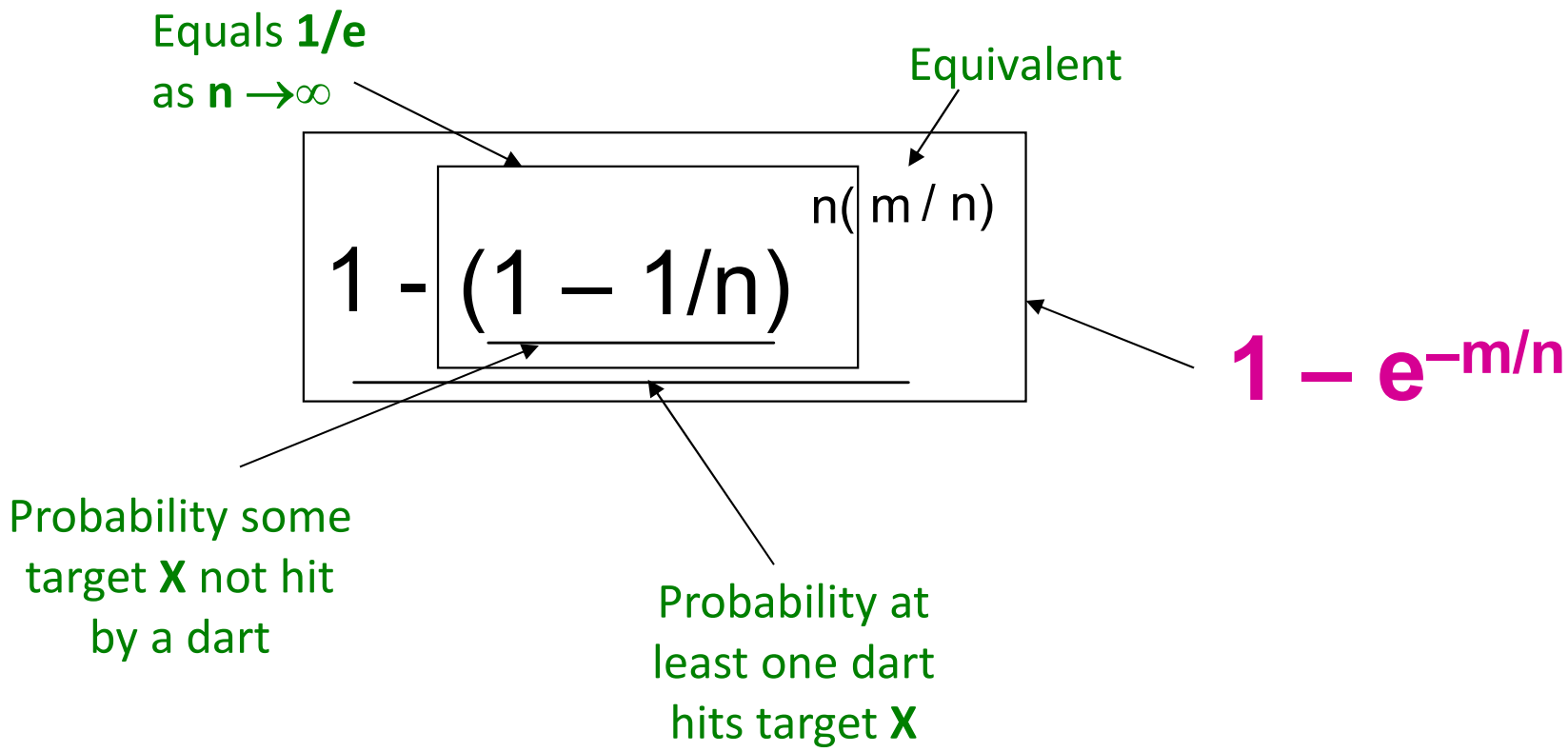
First compute the probability that a target gets no dart, and then take the complement.

P (at least one dart in a target)
= 1 − P (no dart in that target)

Takeaway: Targets = bits in B. Darts = hash values of items in S. False positives occur when a non-member hashes to a bit that is already 1. We analyze this using the darts-targets model.

Analysis: Throwing Darts (2)

- We have ***m*** darts, ***n*** targets
- **What is the probability that a target gets at least one dart?**



Analysis: Throwing Darts (2)

- We have m darts, n targets
- What is the probability that a target gets at least one dart?

Setup / Analogy:

Each dart is thrown independently and lands on one of the n targets with equal probability.

Focus on one particular target X.

(1) Probability that one dart does NOT hit X

$$P(\text{hit } X) = \frac{1}{n}$$

$$\boxed{P(\text{not hit } X) = 1 - \frac{1}{n}}$$

(2) Probability that all m darts miss X

Since darts are independent,

$$\boxed{P(\text{no dart hits } X) = \left(1 - \frac{1}{n}\right)^m}$$

This means the bit corresponding to X remains 0.

(3) Probability that at least one dart hits X

Use complement rule:

$$P(\text{at least one hit}) = 1 - P(\text{no dart hits } X)$$

$$\boxed{P(\text{at least one hit}) = 1 - \left(1 - \frac{1}{n}\right)^m}$$

This is the exact probability that the target X gets at least one dart.

In our filter, this is the probability that a particular bit becomes 1.

(4) Approximation for large n

We use $\left(1 - \frac{1}{n}\right)^n \approx \frac{1}{e} = e^{-1}$ for large n.

Then,

$$\left(1 - \frac{1}{n}\right)^m = \left[\left(1 - \frac{1}{n}\right)^n\right]^{m/n} \approx \left(e^{-1}\right)^{m/n} = e^{-m/n}$$

$$\boxed{\text{So, } P(\text{at least one hit}) \approx 1 - e^{-m/n}} \quad \text{(Useful formula)}$$

Apply to the email example

$m = |S| =$ 1 billion email addresses

$n = |B| =$ 8 billion bits

$$\frac{m}{n} = \frac{1 \text{ billion}}{8 \text{ billion}} = \frac{1}{8}$$

$$P(\text{bit} = 1) \approx 1 - e^{-1/8}$$

$$\approx 1 - 0.8825$$

$$\approx 0.1175 = 11.75\%$$

This is the false positive probability (more accurate!)

Key takeaways:

- Targets = bits in the bit array (n)
- Darts = hash values of items (m)
- $P(\text{false positive}) = P(\text{bit} = 1)$

$$\approx 1 - e^{-m/n}$$

for the first-cut solution (one hash).

Analysis: Throwing Darts (3)

- **Fraction of 1s in the array B =**
  = probability of false positive = $1 - e^{-m/n}$

- **Example:** 10^9 darts, $8 \cdot 10^9$ targets
  - Fraction of **1s** in **B** = $1 - e^{-1/8} = 0.1175$
    - Compare with our earlier estimate: **1/8 = 0.125**

Analysis: Throwing Darts (3)

- From previous slide, we derived that

> Probability that a particular target (bit) gets at least one dart (i.e., becomes 1) $= 1 - (1 - \frac{1}{n})^m$

where
m = number of darts (items inserted)
n = number of targets (bits in array B)

For large n, using $(1 - \frac{1}{n})^n \approx e^{-1}$, we get

> $P(\text{at least one dart hits a target}) \approx 1 - e^{-m/n}$

This is the fraction of 1's in the bit array B. In the filter, this equals the probability of a false positive (for one-hash case).

- Example: 10^9 darts, $8 \cdot 10^9$ targets

Here, $m = 10^9$, $n = 8 \times 10^9$

Compute $\frac{m}{n} = \frac{10^9}{8 \times 10^9} = \frac{1}{8}$

Fraction of 1's in B (= P(false positive)) $= 1 - e^{-m/n} = 1 - e^{-1/8}$

Now, $e^{-1/8} \approx 0.8825$

$\therefore$ P(false positive) $\approx 1 - 0.8825 = 0.1175 = 11.75\%$

- Compare with our earlier rough estimate:

Earlier estimate

Fraction of 1's $\approx \frac{m}{n} = \frac{1}{8} = 0.125 = 12.5\%$

Why the new value is smaller?

Because multiple items may hash to the same bit (collisions). So, fewer than m distinct bits become 1. Hence, actual fraction of 1's $< \frac{m}{n}$.

Takeaway: For the first-cut (single hash) solution, the probability of a false positive is $P(fp) \approx 1 - e^{-m/n}$

Bloom Filter

- Consider: $|S| = m$, $|B| = n$
- Use k independent hash functions $h_1, \ldots, h_k$
- **Initialization:**
  - Set **B** to all **0s**
  - Hash each element $s \in S$ using each hash function h_i, set $B[h_i(s)] = 1$ (for each $i = 1, \ldots, k$)

(**note:** we have a single array B!)

- **Run-time:**
  - When a stream element with key x arrives
    - If $B[h_i(x)] = 1$ for all $i = 1, \ldots, k$ then declare that x is in S
      - That is, x hashes to a bucket set to **1** for every hash function $h_i(x)$
    - Otherwise discard the element x

Bloom Filter

- Consider: $|S| = m$, $|B| = n$
- Use k independent hash functions $h_1, h_2, \ldots, h_k$
- We maintain a single bit array B of size n. (not k different arrays)

Example: $k = 3$

For an element s, suppose:

$h_1(s) = 2$
$h_2(s) = 6$
$h_3(s) = 9$

Then set
$B[2] = B[6] = B[9] = 1$

1) Initialization:

- Set B to all 0s.
- For each element $s \in S$:
  For each hash function h_i, $i = 1, \ldots, k$
  Compute $j = h_i(s)$ and set $B[j] = 1$.

Index →	0	1	2	3	4	5	6	7	8	9	10	...	n-1
B →	0	0	1	0	0	0	1	0	0	1	0	...	0

Only k bits are set to 1 for each element.

2) Run-time (Query):

- When a stream element with key x arrives:
  Compute $j_i = h_i(x)$ for each $i = 1, \ldots, k$
  Check all the k positions $B[j_i]$.

Case A: All k bits are 1

If $B[h_1(x)] = 1$, $B[h_2(x)] = 1$, ..., $B[h_k(x)] = 1$
then declare that x is in S
(x may be in S, could be false positive)

Case B: At least one bit is 0

If $B[h_i(x)] = 0$ for at least one $i \in \{1, \ldots, k\}$
then declare that x is NOT in S
(x is definitely not in S)

Key Property:

- If $x \in S$ → Always reported as "possibly in S" (no false negatives).
- If $x \notin S$ → May be reported as "possibly in S" (false positive), but if even one bit is 0, it is definitely not in S.

0 → Definitely NOT in S
1 → Possibly in S

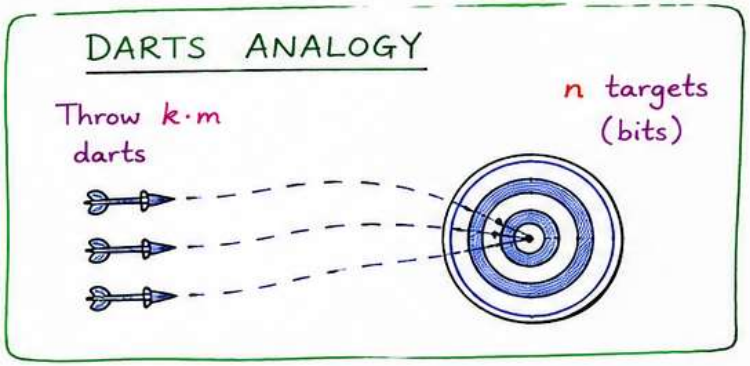
Bloom Filter -- Analysis

- **What fraction of the bit vector B are 1s?**
  - Throwing $k \cdot m$ darts at n targets
  - So fraction of **1**s is $(1 - e^{-km/n})$
- But we have k independent hash functions and we only let the element x through **if all** k hash element x to a bucket of value **1**
- So, false **positive probability** = $(1 - e^{-km/n})^k$

Bloom Filter -- Analysis

- **Q1. What fraction of the bit vector B are 1s ?**
  - We insert m elements and use k independent hash functions.
  - Each element is hashed k times.

Total number of hash outputs $= k \cdot m$



Darts Analogy:

Think of each hash output as a dart.

Darts $= k \cdot m$

Targets $= n$ bits (positions in the array B)

From previous result (one hash case):

$$P(\text{bit} = 0) \approx e^{-(\text{number of darts/targets})}$$

Here, number of darts $= k \cdot m$ and targets $= n$

$$P(\text{bit} = 0) \approx e^{-km/n}$$

Therefore, fraction of bits that are 1:

$$P(\text{bit} = 1) = 1 - P(\text{bit} = 0) = 1 - e^{-km/n}$$

Approximately $1 - e^{-km/n}$ fraction of B are 1s.

Q2. What is the false positive probability ?

Consider a new element x ($x \notin S$).

We compute all k hash values:

$$h_1(x), h_2(x), \ldots, h_k(x)$$

The Bloom filter reports "x is in S" only if all k corresponding bits are 1.

False positive event = all k bits are 1

Assuming the k hash functions are independent, the events of the k positions being 1 are (approximately) independent.

Probability that one position is 1 $\approx 1 - e^{-km/n}$

Therefore,

$$P(\text{false positive}) = \left(1 - e^{-km/n}\right)^k$$

Comparison:

- First-cut (single hash, $k = 1$): $P(FP) \approx 1 - e^{-m/n}$
- Bloom filter (k hashes): $P(FP) \approx \left(1 - e^{-km/n}\right)^k$

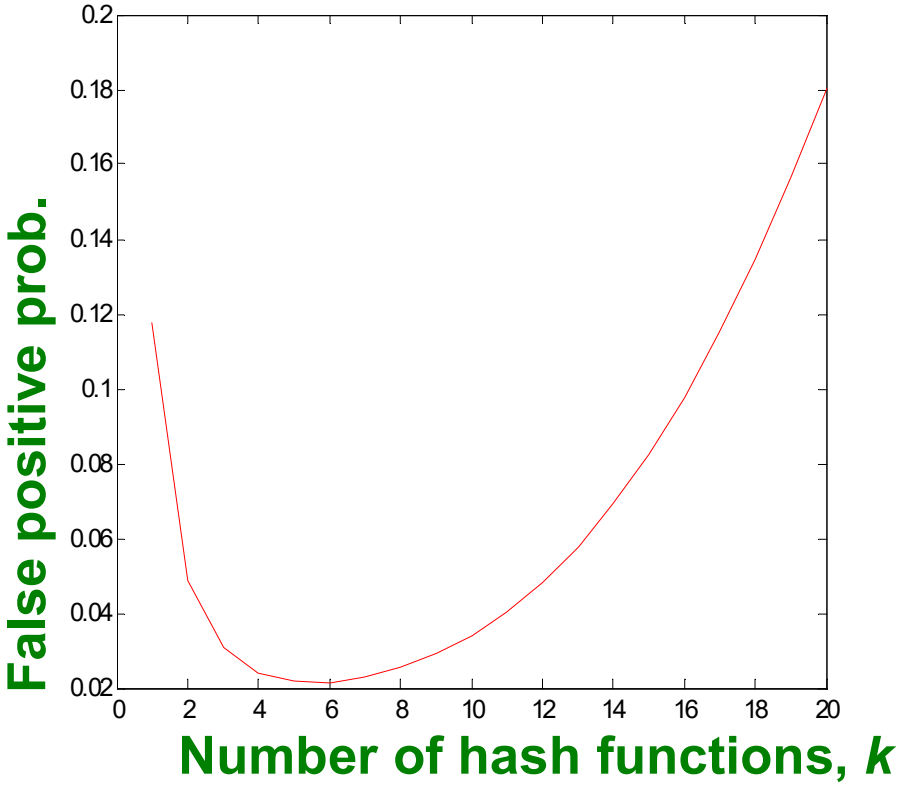
Intuition: With more hashes, a non-member must hit many 1s together, which is unlikely. But too many hashes also make the array too full of 1s. Hence optimal k exists.

Key Takeaways:

1. Total hash outputs $= k \cdot m$ $\Rightarrow$ more darts thrown.
2. More darts set more bits to 1, but also reduce 0s.
3. False positive occurs only when all k bits checked are 1.
4. There is an optimal k (not "the more, the better").

Bloom Filter – Analysis (2)

- ***m* = 1 billion, *n* = 8 billion**
  - **k = 1**: $(1 - e^{-1/8}) = \mathbf{0.1175}$
  - **k = 2**: $(1 - e^{-1/4})^2 = \mathbf{0.0493}$



- **What happens as we keep increasing *k*?**
- "Optimal" value of ***k***: ***n/m* ln(2)**
  - **In our case:** Optimal **k = 8 ln(2) = 5.54 ≈ 6**
    - **Error at k = 6**: $(1 - e^{-1/6})^2 = \mathbf{0.0235}$

Bloom Filter – Analysis (2)

- **Given:** m = 1 billion , n = 8 billion

  So, $\frac{m}{n} = \frac{1}{8}$

- **False positive probability formula:**

$$P(FP) = \left(1 - e^{-km/n}\right)^k$$

(where k = number of hash functions)

- **For $k = 1$:**

$$P(FP) = \left(1 - e^{-1/8}\right)^1 = 1 - e^{-1/8} = 0.1175$$

$$= 11.75\ \%$$

- **For $k = 2$:**

$$P(FP) = \left(1 - e^{-2/8}\right)^2 = \left(1 - e^{-1/4}\right)^2 = 0.0493$$

$$= 4.93\ \%$$

- **What happens as we keep increasing k ?**

Initially, as k increases, P(FP) decreases because a non-member has to pass more checks (all k bits must be 1). But after some point, too many hashes set many bits to 1, the array becomes crowded, and P(FP) starts increasing. Hence the graph is U-shaped.

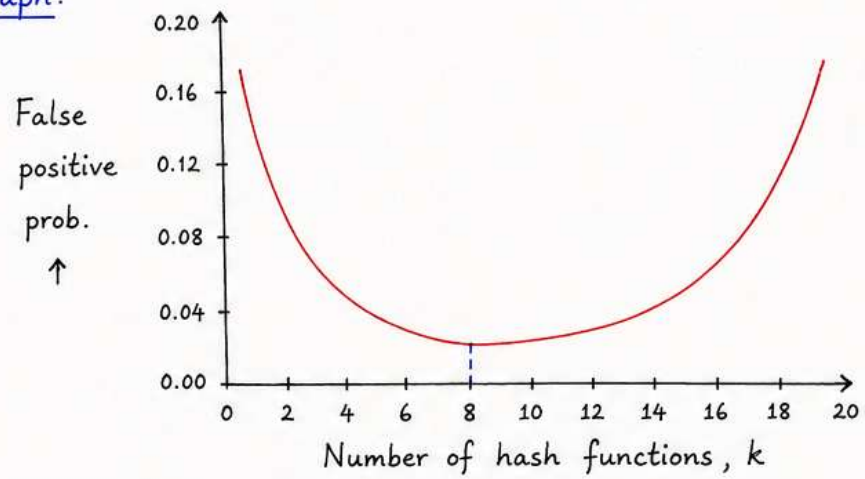
Intuition:

Small k → few bits set to 1 → easy to pass → high FP

Optimal k → balance → minimum FP

Large k → many bits set to 1 → array too full → high FP

Graph:



- **Optimal value of k:**

The optimal number of hash functions that minimizes P(FP) is:

$$k_{opt} = \frac{n}{m} \ln(2)$$

In our case: $k_{opt} = \frac{n}{m} \ln(2) = 8 \ln(2) \doteq 5.54 \approx 6$

- **Take $k \doteq 6$ (nearest integer):**

$$P(FP) = \left(1 - e^{-6/8}\right)^6 = \left(1 - e^{-0.75}\right)^6 \doteq 0.0235$$

$$= 2.35\ \%$$

Takeaways:

1. Increasing k first reduces false positives.
2. After a point, more k makes the array too full → FP increases.
3. Use optimal $k = \left(\frac{n}{m}\right) \ln 2$ for minimum false positives.
4. In our example (m = 1B, n = 8B): optimal $k \approx 6$, FP ≈ 2.35%.

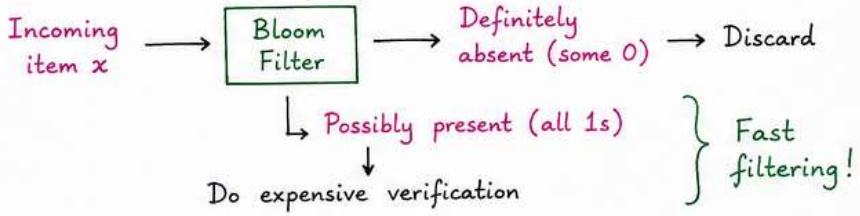
Bloom Filter: Wrap-up

- **Bloom filters guarantee no false negatives, and use limited memory**
  - Great for pre-processing before more expensive checks
- **Suitable for hardware implementation**
  - Hash function computations can be parallelized
- Is it better to have **1** big **B** or ***k*** small **B**s?
  - **It is the same:** $(1 - e^{-km/n})^k$ vs. $(1 - e^{-m/(n/k)})^k$
  - But keeping **1 big B** is simpler

Bloom Filter: Wrap-up

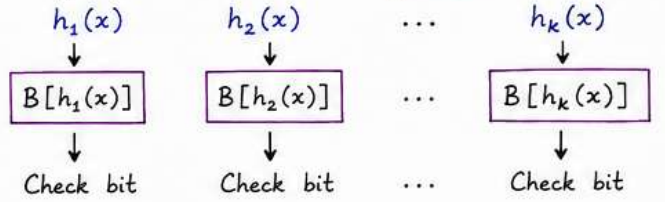
1) No false negatives, limited memory

- If an element x is in S, Bloom filter will never say it is absent. (No false negatives)
- But it may say "possibly present" for some elements not in S. (False positives possible)
- So Bloom filter is great as a pre-processing step before more expensive checks.



2) Suitable for hardware implementation

- We use k hash functions $h_1(x), h_2(x), \ldots, h_k(x)$.
- The k hash computations and the k memory lookups can be done in parallel.



- This makes Bloom filters very suitable for high-speed hardware and networking applications.

3) Is it better to have 1 big B or k small Bs?

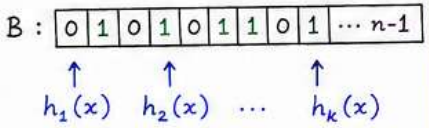
(A) One big Bloom filter (1 array B of size n bits)

- Use k hash functions.
- All hash values go into the same array B.
- False positive probability:

$$P(FP) = \left(1 - e^{-km/n}\right)^k$$

(we derived earlier)

One big array B



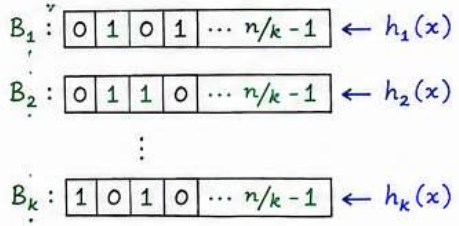
(B) k smaller Bloom filters (k arrays)

- Split the memory into k arrays.
- Each array size = n/k bits.
- Use one hash function for each array.
- False positive probability:

$$P(FP) = \left(1 - e^{-m/(n/k)}\right)^k = \left(1 - e^{-km/n}\right)^k$$

(Same as case A)

k small arrays



Conclusion: The theoretical false positive probability is the same in both designs, assuming independent hash functions.

Key Takeaways:

1. Bloom filters give no false negatives but may have false positives.
2. They are memory efficient and fast – great for filtering before expensive checks.
3. Hash computations can be parallelized → good for hardware.
4. Using k hash functions, FP probability = $\left(1 - e^{-km/n}\right)^k$ and optimal $k = \left(\frac{n}{m}\right) \ln 2$.
5. One big array B is usually simpler and is preferred in practice.

Why prefer 1 big B?

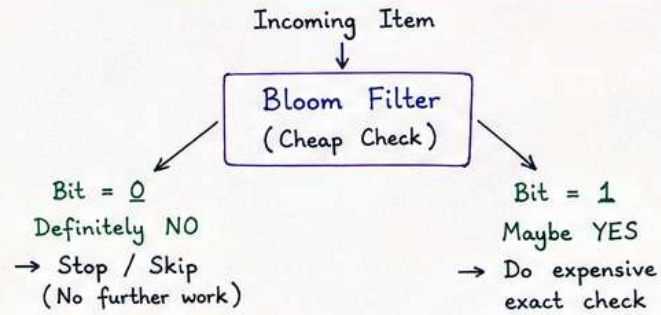
- Implementation is simpler.
- Only one array to maintain and update.
- Cache-friendly and easy to manage.

BLOOM FILTER – Applications in Today's World

Bloom Filter is a memory-efficient data structure that answers the question:
"Could this item belong to a large set?"

0 = Definitely NO
1 = Maybe YES

How it is used (General pattern)



1. Databases & Storage Systems

- Question: "Could this key exist in this disk / SSTable block?"
- Without Bloom Filter → many unnecessary disk reads.

Example: GET customer_827361 → Bloom Filter
- 0 → Key definitely not present in this block. (Avoid disk read)
- 1 → Key may be present (Do actual lookup)

2. Web Crawlers

- Billions of URLs to crawl.
- Check: "Have we possibly visited this URL before?"

Example: URL → Bloom Filter
- 0 → Definitely not seen → Crawl it
- 1 → Maybe seen → Do additional check

3. Cache Protection / Avoiding Unnecessary Queries

- Many requests are for keys that don't even exist.
- Prevent unnecessary cache / database lookups.

Flow: Request → Bloom Filter
- 0 → Don't query (Key definitely absent)
- 1 → Check Cache / DB

4. Cybersecurity & Malicious URL Screening

- Maintain a huge set of known bad domains, URLs, hashes.
- Fast first-stage screening of incoming requests.

Flow: Incoming URL / Domain → Bloom Filter
- 0 → Not in bad set (Allow / Continue)
- 1 → Possibly in bad set (Do deeper security check)

5. Duplicate Detection

- Millions of documents, files, events arrive.
- Check: "Have we possibly seen this before?"

Example: Incoming Document → hash → Bloom Filter
- 0 → New item
- 1 → Possibly duplicate (Do exact compare)

6. Recommendation & Content Systems

- For each user, keep a compact filter of items already shown.
- Before recommending an item:

Candidate Item → Bloom Filter
- 0 → Not shown before (Good candidate)
- 1 → Possibly shown before (Check history if needed)

7. Blockchain / Distributed Systems

- Clients may be interested only in specific addresses / patterns.
- Bloom filter helps quickly detect transactions that MAY be relevant.
- Saves bandwidth and processing.

In all cases, Bloom Filter acts as a fast, memory-efficient pre-filter before expensive exact processing.

Key Takeaway: Bloom Filter never gives false negatives. It may give false positives, but that is acceptable in many real-world systems where speed & memory matter most.

Common Applications (Summary):
Databases & Storage → Web Crawling → Caching → Security → Deduplication / Stream Processing → More.

(2) Counting Distinct Elements

Counting Distinct Elements

- **Problem:**
  - Data stream consists of a universe of elements chosen from a set of size ***N***
  - Maintain a count of the number of distinct elements seen so far
- **Obvious approach:**

  Maintain the set of elements seen so far
  - That is, keep a hash table of all the distinct elements seen so far

Counting Distinct Elements

① Problem :

- We are given a data stream. Each element in the stream is chosen from a universe of size N.
- We want to maintain a count of the number of distinct elements seen so far.

Universe of elements :

$U = \{ e_1, e_2, e_3, \ldots, e_N \}$ where $|U| = N$

Example :

Suppose stream is :

A, B, A, C, B, D, A, E, C, F, D

Distinct elements = $\{ A, B, C, D, E, F \}$

Distinct count = 6

② Obvious approach :

Maintain the set of elements seen so far.

- That is, keep a hash table (or set) of all the distinct elements seen so far.
- When a new element x from the stream arrives :
  - ① If x is not in the set → insert it.
  - ② If x is already in the set → do nothing.

At any point of time, Distinct count = size of the set

Example :

Incoming element	A	B	A	C	B	D	A	E
Set of distinct elements seen so far	{A}	{A,B}	{A,B}	{A,B,C}	{A,B,C}	{A,B,C,D}	{A,B,C,D}	{A,B,C,D,E}
Distinct count	1	2	2	3	3	4	4	5

Set representation (Hash table) :

Hash index	
0	C
1	A
2	D
3	E
⋮	
m-1	B

Memory requirement :
Proportional to the number of distinct elements seen so far.

Problem with this approach :
If the number of distinct elements is very large (millions or billions), this approach needs a lot of memory.

Can we estimate the number of distinct elements using much less memory ?

Applications

- **How many different words are found among the Web pages being crawled at a site?**
  - Unusually low or high numbers could indicate artificial pages (spam?)
- **How many different Web pages does each customer request in a week?**
- **How many distinct products have we sold in the last week?**

Applications

Counting distinct elements is useful in many real-world situations.
In a data stream, we want to know the number of unique (different) items.

(1) How many different words are found among the web pages being crawled at a site?

- Search engine crawls pages and sees words continuously.

Example stream of words :

AI, data, model, AI, learning, data, cloud, model, web, AI, cloud

Distinct words = { AI, data, model, learning, cloud, web }

Distinct count = 6

If the number of different words on a site is unusually low or unusually high, it may indicate artificial or spam-like pages.

---

(2) How many different web pages does each customer request in a week?

- Count unique pages requested by one customer during a week.

Example stream of page requests :

Page A, Page B, Page A, Page C, Page A, Page B, Page D, Page C

Distinct pages = { Page A, Page B, Page C, Page D }

Distinct page count for this customer = 4

Why useful?

- Measure user interest diversity.
- Find power users visiting many different pages.
- Detect unusual user behaviour.

---

(3) How many distinct products have we sold in the last week?

- In sales data stream, same product can be sold many times.

Example stream of sold products :

Laptop, Mouse, Laptop, Keyboard, Mouse, Phone, Laptop, Headphones, Phone

Distinct products = { Laptop, Mouse, Keyboard, Phone, Headphones }

Distinct product count in last week = 5

Why useful?

- Understand product variety.
- Inventory and stock planning.
- Detect new / active products.
- Business analytics.

⇒ In all these applications, the stream is very large and storing every distinct item is expensive.
So we need algorithms (like FM algorithm, etc.) to estimate the number of distinct elements using much less memory.

Using Small Storage

- **Real problem: What if we do not have space to maintain the set of elements seen so far?**
- **Estimate the count in an unbiased way**
- **Accept that the count may have a little error, but limit the probability that the error is large**

Using Small Storage

■ 1) Real problem : What if we do not have space to maintain the set of elements seen so far?

- In a data stream, the number of distinct elements can be huge.

Example :

Stream of user IDs → 101, 205, 101, 450, 205, 700, 900, 101, 1200, ...

Distinct elements seen so far = {101, 205, 450, 700, 900, 1200, ...}

⇒ If there are millions or billions of distinct elements, storing all of them (using a hash table) requires huge memory.

2) Estimate the count in an unbiased way

- Instead of exact count D, we compute an estimate $\hat{D}$.
- The estimate should be unbiased, i.e.,

$$E[\hat{D}] = D$$

(Expected value of estimate should be equal to the true distinct count D)

Example : If the true count $D = 100$, then repeated runs of our algorithm may give estimates like

95, 103, 98, 105, 99, ...

Some are above 100, some below 100, but on average they should be close to 100.

3) Accept that the count may have a little error, but limit the probability that the error is large

- We allow some error in the estimate.

Notation :
D = true number of distinct elements
$\hat{D}$ = estimated count

- Suppose $D = 10{,}000$. An estimate like $\hat{D} = 10{,}200$ (error = 200) is acceptable.

But an estimate like $\hat{D} = 5{,}000$ (error = 5,000) is a large error and should happen with very small probability.

- Our goal is to design an algorithm such that for any given error parameter $\varepsilon > 0$, the estimate satisfies

$$P(|\hat{D} - D| > \varepsilon D) \leq \delta$$

where δ $(0 < \delta < 1)$ is a small number (failure probability).

Main idea : We trade a small amount of accuracy for a huge saving in memory.

Less memory ⟺ Approximate answer

This is the basic philosophy behind streaming algorithms like FM (Flajolet-Martin).

Flajolet-Martin Approach

- Pick a hash function ***h*** that maps each of the ***N*** elements to at least $\log_2 N$ bits
- For each stream element ***a***, let ***r(a)*** be the number of trailing **0s** in ***h(a)***
  - **r(a)** = position of first 1 counting from the right
    - E.g., say ***h(a) = 12***, then ***12*** is ***1100*** in binary, so ***r(a) = 2***
- Record ***R* = the maximum *r(a)* seen**
  - $\mathbf{R = max_a\ r(a)}$, over all the items ***a*** seen so far
- **Estimated number of distinct elements = 2^R**

Flajolet-Martin Approach

(1) Pick a hash function h that maps each of the N elements to at least $\log_2 N$ bits.

- The hash function should distribute values uniformly.
- It converts every element to a binary number of length at least $\log_2 N$ bits.

Why $\log_2 N$ bits?

- With $\log_2 N$ bits we have $2^{\log_2 N} = N$ possible values, so collisions are minimized.

(2) For each stream element a, let $r(a)$ be the number of trailing 0s in $h(a)$.

- $r(a)$ = position of first 1 counting from the right (starting position 0).

Example: $h(a) = 12_{10} = 1100_2$

two trailing 0s $\Rightarrow$ $r(a) = 2$

(3) Record R = the maximum $r(a)$ seen so far.

- $R = \max_a r(a)$ over all the items a seen so far.

(4) Estimated number of distinct elements = 2^R

- If R is large, it indicates we have seen many distinct elements.
- If R is small, then number of distinct elements is small.

Example: Assume we use a hash function h and see the following stream elements.

Stream: A, B, C, A, D, E, F, G

Element (a)	A	B	C	A	D	E	F	G
h(a) (decimal)	12	10	7	12	8	20	5	24
h(a) (binary)	1100	1010	0111	1100	1000	10100	0101	11000
r(a) = # trailing 0s	2	1	0	2	3	2	0	3

Maximum $r(a)$ seen so far $(R) = \max\{2, 1, 0, 2, 3, 2, 0, 3\} = 3$

Estimated number of distinct elements = $2^R = 2^3 = 8$

Why does this work?

For a random binary hash value,

At least R trailing 0s	Probability	Meaning
$R \geq 0$	1	Always true
$R \geq 1$	$1/2$	Ends with ≥ 1 zero
$R \geq 2$	$1/4$	Ends with ≥ 2 zeros
$R \geq 3$	$1/8$	Ends with ≥ 3 zeros
$R \geq R$	$1/2^R$	R is large $\Rightarrow$ rare event

Seeing a value with R trailing zeros is a $1/2^R$ event.

So we need about 2^R distinct elements to see it with good chance.

Intuition: More distinct elements $\Rightarrow$ more chances to see a rare event (many trailing 0s) $\Rightarrow$ larger R.

Note: This gives an estimate, not exact count. To reduce error, we use multiple hash functions and take the average.

Why It Works: Intuition

- **Very very rough and heuristic intuition why Flajolet-Martin works:**
  - ***h(a)*** hashes ***a*** with **equal prob.** to any of ***N*** values
  - Then ***h(a)*** is a sequence of $\mathbf{log_2 N}$ bits, where 2^{-r} fraction of all ***a***s have a tail of ***r*** zeros
    - About 50% of ***a***s hash to ***0**
    - About 25% of ***a***s hash to ****00**
    - So, if we saw the longest tail of ***r=2*** (i.e., item hash ending ***100**) then we have probably seen **about *4*** distinct items so far
  - **So, it takes to hash about 2^r items before we see one with zero-suffix of length *r***

Why It Works: Intuition

▶ Very very rough and heuristic intuition why Flajolet-Martin works:

① $h(a)$ hashes a with equal probability to any of N values.

- In other words, h is a uniform hash function.

② Then $h(a)$ is a sequence of $\log_2 N$ bits.

Let r = number of trailing 0s in $h(a)$.

- For a random bit sequence, the probability that it ends with r zeros is:

$$P(r \text{ trailing 0s}) = \frac{1}{2^r}$$

Some specific cases:

P(ends with at least 1 zero) = 1/2 ≈ 50% (**0)

P(ends with at least 2 zeros) = 1/4 ≈ 25% (**00)

P(ends with at least 3 zeros) = 1/8 ≈ 12.5% (*000)

⋮

$P(\text{ends with at least } \bar{r} \text{ zeros}) = \frac{1}{2^r}$

Key idea: Longer tail of zeros is a rarer event.
We need more distinct items to see such a rare event.

③ What does this mean?

- If the longest tail of zeros observed so far is r, it means we have seen a hash value ending with r zeros.
- But such an event happens with probability $1/2^r$.
- So, roughly speaking, we need about 2^r distinct items before we expect to see one item whose hash value has r trailing zeros.

Example: Suppose we see a hash value ending with $r = 2$ trailing zeros, i.e., pattern *100.

$$P(\text{such pattern}) = \frac{1}{2^2} = \frac{1}{4} \quad (\approx 25\%)$$

Therefore, we have probably seen about $2^2 = 4$ distinct items so far.

④ Hence, if we observe a maximum of r trailing 0s in all hash values seen so far, then

Estimated number of distinct elements ≈ 2^r

Intuition in one line:
More distinct elements ⟹ more chances to see a rare event (many trailing 0s) ⟹ larger r.

Why It Works: More formally

- **Now we show why Flajolet-Martin works**
- **Formally, we will show that probability of finding a tail of *r* zeros:**
  - **Goes to 1 if $m \gg 2^r$**
  - **Goes to 0 if $m \ll 2^r$**

  where m is the number of distinct elements seen so far in the stream
- **Thus, 2^R will almost always be around *m!***

Why It Works: More formally

- Now we show why Flajolet-Martin works.
- Recall: $R = \max r(a)$ = maximum number of trailing 0s seen in the hash values of all distinct elements so far.

Definition: $r(a)$ = number of trailing 0s in the binary representation of $h(a)$.

Key Probability Fact:

* For a random binary number,

$$P(\text{at least } r \text{ trailing 0s}) = \frac{1}{2^r}$$

Why? For each extra trailing 0 we need the next bit to be 0, which happens with probability $1/2$.

Examples:

$P(\text{at least 1 trailing 0}) = 1/2 = 50\%$ (... *0)

$P(\text{at least 2 trailing 0s}) = 1/4 = 25\%$ (... **00)

$P(\text{at least 3 trailing 0s}) = 1/8 = 12.5\%$ (... ***000)

⋮

Main Idea: Among m distinct elements, seeing a long suffix of 0s is a very event.

⇒ The longer the tail we observe, the larger is m.

Let m = number of distinct elements seen so far.

What is the probability that we see at least r trailing 0s in the hash value of one of these m elements?

Expected number of elements with $\geq r$ trailing 0s among m elements $= m \times \left(\frac{1}{2^r}\right) = \frac{m}{2^r}$

- **Goes to 1 if $m \gg 2^r$**

  If m is much larger than 2^r, then $m/2^r$ is very large. We expect many elements to have $\geq r$ trailing 0s. So probability of finding such a tail goes to 1 (almost sure).

- **Goes to 0 if $m \ll 2^r$**

  If m is much smaller than 2^r, then $m/2^r$ is very small. It is very unlikely to see even one element with r trailing 0s. So probability goes to 0.

Therefore, the transition happens around $m \approx 2^r$. If $m \approx 2^r$, we have a reasonable chance (neither very small nor almost 1) to see a tail of length r.

Conclusion: Since the largest number of trailing 0s observed so far is R, we conclude that

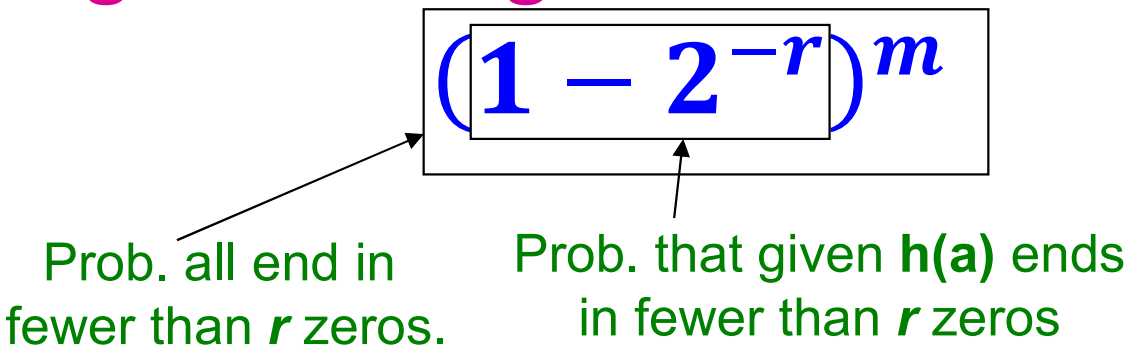
m (true # of distinct elements) is around 2^R

So, Flajolet-Martin estimator = 2^R

Intuition in one line: Finding a tail of r zeros is a 1 in 2^r event. If we see it, we must have tried about 2^r distinct items.

Why It Works: More formally

- **What is the probability that a given *h*(*a*) ends in at least *r* zeros is 2^{-r}**
  - **h(a)** hashes elements uniformly at random
  - Probability that a random number ends in at least ***r*** zeros is $\mathbf{2^{-r}}$
- Then, the probability of **NOT** seeing a tail of length ***r*** among ***m*** elements:



Why It Works: More formally

- **Recall:** We use a hash function h that maps each stream element to a random binary number.

> Let m = number of distinct elements seen so far.
> Let r = a fixed non-negative integer.

(1) Probability that a given $h(a)$ ends in at least r zeros

- $h(a)$ hashes elements uniformly at random.
- For a random binary number, the probability that it ends in at least r zeros is 2^{-r}.

> Therefore,
> $P(h(a)$ has at least r trailing 0s$) = 2^{-r}$

Examples:

$r = 0 \rightarrow P(\text{at least 0 zeros}) = 2^0 = 1$ (always true)
$r = 1 \rightarrow P(\text{at least 1 zero}) = 2^{-1} = 1/2$
$r = 2 \rightarrow P(\text{at least 2 zeros}) = 2^{-2} = 1/4$
$r = 3 \rightarrow P(\text{at least 3 zeros}) = 2^{-3} = 1/8$
$\vdots$

(2) Probability of NOT seeing a tail of length r among m elements

For one element, the probability that it has fewer than r trailing 0s is (by complement rule):

> $P(\text{less than } r \text{ trailing 0s}) = 1 - 2^{-r}$

Conclusion: As m increases, $(1-2^{-r})^m$ becomes smaller and smaller.

$\Longrightarrow$ The probability of seeing a tail of at least r zeros increases.

Among all m distinct elements, we want the probability that none of them has r or more trailing zeros.

$$P(\text{no tail of length } r \text{ among } m \text{ elements}) = (1-2^{-r})^m$$

- $(1-2^{-r})$: Prob. all end in fewer than r zeros
- $(1-2^{-r})$: Prob. that a given $h(a)$ ends in fewer than r zeros
- m: m elements

Derivation:

- For each element, $P(\text{fewer than } r \text{ zeros}) = 1 - 2^{-r}$
- Assuming hash values behave independently,
- Probability that all m elements have fewer than r zeros
  $= (1-2^{-r}) \times (1-2^{-r}) \times \cdots \times (1-2^{-r})$ (m times)
  $= (1-2^{-r})^m$

Example: Suppose $r = 3$ and $m = 10$.

For one element:
$P(\text{at least 3 zeros}) = 2^{-3} = 1/2$
$\Rightarrow P(\text{less than 3 zeros}) = 1 - 1/8 = 7/8$

Probability that none of the 10 elements has 3 or more zeros:

$$\left(\frac{7}{8}\right)^{10} \approx 0.263 \quad (\approx 26.3\%)$$

> Thus, probability that at least one element has ≥ 3 zeros
> $= 1 - \left(\frac{7}{8}\right)^{10} \approx 0.737 \quad (\approx 73.7\%)$

This is the key reason why 2^R (where R = max trailing zeros seen) gives a good estimate of m.

Why It Works: More formally

- **Note:** $(1-2^{-r})^m = (1-2^{-r})^{2^r (m2^{-r})} \approx e^{-m2^{-r}}$
- **Prob. of NOT finding a tail of length *r* is:**
  - If ***m* << 2^r^**, then prob. tends to **1**
    - $(1-2^{-r})^m \approx e^{-m2^{-r}} = 1$ as **m/2^r^→ 0**
    - So, the probability of finding a tail of length ***r*** tends to **0**
  - If ***m* >> 2^r^**, then prob. tends to **0**
    - $(1-2^{-r})^m \approx e^{-m2^{-r}} = 0$ as **m/2^r^ → ∞**
    - So, the probability of finding a tail of length ***r*** tends to **1**
- **Thus, 2^R will almost always be around *m!***

Why It Works: More formally

From previous slide we had:

$$P(\text{NOT seeing a tail of length } r) = (1 - 2^{-r})^m$$

where, m = number of distinct elements seen so far

r = a fixed non-negative integer

Note: $(1-x)^n = (1-x)^{2^r \cdot \frac{n}{2^r}} = \left((1-x)^{2^r}\right)^{\frac{m}{2^r}}$

For small x, $(1-x)^{2^r} \approx e^{-x \cdot 2^r}$ (standard approximation)

So, using this with $x = 2^{-r}$,

$$\begin{aligned}(1 - 2^{-r})^m &= \left((1 - 2^{-r})^{2^r}\right)^{\frac{m}{2^r}} \\ &\approx \left(e^{-2^{-r} \cdot 2^r}\right)^{\frac{m}{2^r}} \\ &= \left(e^{-1}\right)^{\frac{m}{2^r}} \\ &= e^{-m/2^r}\end{aligned}$$

Therefore,

$$P(\text{NOT seeing a tail of length } r) \approx e^{-m/2^r}$$

This probability depends only on the ratio $\frac{m}{2^r}$

Intuition:

- A long tail of zeros is a rare event.
- As the number of distinct elements increases, chances of seeing a long tail increase.

Prob. of NOT finding a tail of length r is:

Case 1: If $m \ll 2^r$ (m is much smaller than 2^r)

Then $\frac{m}{2^r} \to 0$

So, $P(\text{NOT seeing a tail of length } r) \approx e^{-m/2^r} \approx e^0 = 1$

i.e., probability tends to 1.

$\Rightarrow$ It is very unlikely to see a tail of length r (probability of seeing such a tail tends to 0).

Case 2: If $m \gg 2^r$ (m is much larger than 2^r)

Then $\frac{m}{2^r} \to \infty$

So, $P(\text{NOT seeing a tail of length } r) \approx e^{-m/2^r} \approx e^{-\infty} = 0$

i.e., probability tends to 0.

$\Rightarrow$ It is very likely to see a tail of length r (probability of seeing such a tail tends to 1).

Transition happens around $\frac{m}{2^r} \approx 1$

i.e., $m \approx 2^r$

So, 2^r will almost always be around m!

Conclusion:

Let R = max number of trailing zeros observed.

Then $m \approx 2^R$ (FM estimate)

Why It Doesn't Work

- **$E[2^R]$ is actually infinite**
  - Probability halves when ***R*** $\rightarrow$ ***R+1***, but value doubles
- **Workaround involves using many hash functions h_i and getting many samples of R_i**
- **How are samples R_i combined?**
  - **Average?** What if one very large value 2^{R_i}?
  - **Median?** All estimates are a power of **2**
  - **Solution:**
    - Partition your samples into small groups
    - Take the median of groups
    - Then take the average of the medians

Why It Doesn't Work

- **The basic FM estimator:**

$$\hat{m} = 2^R$$

where R = maximum number of trailing zeros observed.

- **Problem 1:** $E[2^R]$ is actually infinite
  - If $R \to R+1$, the estimate $2^R \to 2^{R+1} = 2(2^R)$ (value doubles).
  - But the probability of seeing such a large R roughly halves:

$$P(R = r) \propto 2^{-r}$$

> So, probability halves but value doubles.
> This creates a heavy tail → very large values occur.

R	2^R
5	32
6	64
10	1024
20	1,048,576
...	...

⇒ In the idealized unbounded model, $E[2^R] = \infty$.

- **Problem 2:** Using one hash function is too noisy
  - A single 2^R can be extremely large due to one rare event.

One run:

R	5	5	6	5	11	5	6	...
2^R	32	32	64	32	2048	32	64	...

one rare large value distorts the result

- Average? Very sensitive to such rare huge values.
- Median? Better (robust to outliers).
  But all estimates are powers of 2 $(1, 2, 4, 8, 16, \ldots)$.
  So simple median is coarse: jumps between 32, 64, 128, ...

- **Workaround: Use many hash functions**
  - Use k independent hash functions $h_1, h_2, \ldots, h_k$.
  - For each h_i, compute its own maximum trailing zeros R_i and estimate 2^{R_i}.

Example: | 32 | | 64 | | 32 | | 64 | | 1024 | | 32 | | 64 | | 32 | | 128 | ... (from different hash functions)

How to combine these samples R_i ?

- **Solution (Robust combination)**

① Partition your samples into small groups.

② Take the median of each group.

③ Then take the average of these medians (of groups).

Many samples R_i
↓
Divide into groups (small size g)
↓
Median in each group
↓
Average of medians

Intuition

- Group median removes the effect of extreme values within each group.
- Averaging the medians smoots the result and uses all groups.

Key takeaway

One FM estimator 2^R is very high variance.
Using many hash functions and combining via "median of groups, then average" gives a stable and accurate estimate.

Result: The final estimate is close to the true number of distinct elements m.

Remember: 2^R is a good idea, but one is not enough!

(3) Computing Moments

Generalization: Moments

- Suppose a stream has elements chosen from a set *A* of *N* values
- Let m_i be the number of times value *i* occurs in the stream
- The kth *moment* is

$$\sum_{i \in A} (m_i)^k$$

Generalization : Moments

- Suppose a stream has elements chosen from a set A of N values.
- Let m_i be the number of times value i occurs in the stream.
- The kth moment of the stream is

$$F_k = \sum_{i \in A} (m_i)^k$$

where A = set of all possible values (universe)
and m_i = frequency (count) of element i in the stream.

Intuition :
For each distinct element i, take its frequency m_i, raise it to power k, and add over all elements.
Different k capture different aspects of the stream.

Example : Let $A = \{a, b, c, d\}$

Suppose the stream is : a, b, a, c, a, b, d, b

Frequencies (m_i) :

Element i	a	b	c	d
Frequency m_i	3	3	1	1

Check : Total stream length = 3 + 3 + 1 + 1 = 8

Different moments :

① 0th moment $(k = 0)$ → Number of distinct elements

$$F_0 = \sum_i m_i^0 = \text{number of } i \text{ with } m_i > 0 = 1 + 1 + 1 + 1 = \underline{4}$$

So, F_0 gives the distinct count.

② 1st moment $(k = 1)$ → Total number of stream elements

$$F_1 = \sum_i m_i = 3 + 3 + 1 + 1 = \underline{8}$$

So, F_1 gives the stream length.

③ 2nd moment $(k = 2)$

$$F_2 = \sum_i m_i^2 = 3^2 + 3^2 + 1^2 + 1^2 = 9 + 9 + 1 + 1 = \underline{20}$$

Frequently occurring items get more weight (squared).

④ 3rd moment $(k = 3)$

$$F_3 = \sum_i m_i^3 = 27 + 27 + 1 + 1 = \underline{56}$$

Observation :
As k increases, higher frequencies dominate more because $(m_i)^k$ grows fast when m_i is large.

Summary :
- F_0 = number of distinct elements (connects to FM algorithm)
- F_1 = total number of elements in the stream
- $F_2, F_3, \ldots$ capture heavier repetition / frequency information
- Moments provide a compact summary of the frequency distribution.

Goal in stream mining : Can we estimate F_k for $k \geq 2$ using small memory ?
→ Yes ! This leads to algorithms like AMS (Alon-Matias-Szegedy) algorithm.

Special Cases

$$\sum_{i \in A} (m_i)^k$$

- **0th moment =** number of distinct elements
  - The problem just considered
- **1st moment =** count of the numbers of elements = length of the stream
  - Easy to compute
- **2nd moment =** ***surprise number S*** **=** a measure of how uneven the distribution is

Surprise Number (Second Moment)

- The 2nd moment ($k = 2$) is called the "Surprise Number" S.

General formula:

$$F_2 = \sum_{i \in A} (m_i)^2$$

where m_i = frequency of element i.

Intuition: We square the frequencies. So, elements that appear many times contribute a lot more, while rare elements contribute very little. Therefore, F_2 captures how UNEVEN the distribution is.

Examples: (Both streams have the same length = 4)

Stream A → Evenly distributed

Stream: A, B, C, D

Element i	A	B	C	D
Frequency m_i	1	1	1	1
m_i^2	$1^2 = 1$	$1^2 = 1$	$1^2 = 1$	$1^2 = 1$

$F_2 = 1 + 1 + 1 + 1 = \underline{\underline{4}}$

(Small Surprise Number)

Distribution is uniform.

Stream B → Highly concentrated

Stream: A, A, A, A

Element i	A
Frequency m_i	4
m_i^2	$4^2 = 16$

$F_2 = \underline{\underline{16}}$

(Large Surprise Number)

One element dominates the stream.

Same length ($F_1 = 4$) but very different F_2 values:

4 vs 16

What does Surprise Number tell us?

- Small F_2 → frequencies are spread out (many items occur roughly the same number of times).
- Large F_2 → a few items occur very frequently (distribution is skewed / uneven).

Takeaway: F_2 is a powerful summary of the unevenness of the data. Higher the value, more surprising / concentrated the stream is!

Example: Surprise Number

- **Stream of length 100**
- **11 distinct values**

- Item counts: **10, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9**
  Surprise *S* = 910

- Item counts: **90, 1, 1, 1, 1, 1, 1, 1 ,1, 1, 1**
  Surprise *S* = 8,110

Example: Surprise Number (F_2)

- Stream length = 100
- Distinct values = 11
- Reminder: Surprise Number (second moment)

$$S = F_2 = \sum_i m_i^2$$

where m_i = frequency of the ith item.

(1) **Case 1:** Almost uniform distribution

Item counts: 10, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9 (11 items)

Check length: $10 + 10 \times 9 = 10 + 90 = 100$

$$S = 10^2 + 10(9^2)$$
$$= 100 + 10(81)$$
$$= 100 + 810$$
$$S = 910$$

All items occur roughly equally. So distribution is quite uniform. Surprise is small.

(2) **Case 2:** Highly uneven distribution

Item counts: 90, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 (11 items)

Check length: $90 + 10 \times 1 = 90 + 10 = 100$

$$S = 90^2 + 10(1^2)$$
$$= 8100 + 10$$
$$S = 8{,}110$$

One item dominates the stream. Very uneven distribution. Surprise is large.

What this example shows:

- Both streams have the same $F_0 = 11$ distinct values.
- Both streams have the same $F_1 = 100$ (length).
- But the Surprise Number F_2 is very different: 910 vs 8,110.

Key takeaway: The more concentrated or uneven the frequencies are, the larger the Surprise Number (F_2).

[Alon, Matias, and Szegedy]

AMS Method

- **AMS method works for all moments**
- **Gives an unbiased estimate**
- **We will just concentrate on the 2nd moment *S***
- **We pick and keep track of many variables *X:***
  - For each variable ***X*** we store ***X.el*** and ***X.val***
    - ***X.el*** corresponds to the item ***i***
    - ***X.val*** corresponds to the **count** of item ***i***
  - Note this requires a count in main memory, so number of ***X***s is limited
- **Our goal is to compute $S = \sum_i m_i^2$**

AMS Method [Alon, Matias, and Szegedy]

- AMS (Alon-Matias-Szegedy) method is a sampling based method used to estimate frequency moments in data streams.
- It works for all moments ($F_k = \sum_i m_i^k$).
- It gives an unbiased estimate. i.e., the expected value of the estimate equals the true value of the moment.
- Here we will focus on the 2nd moment (Surprise Number):

$$S = F_2 = \sum_i m_i^2$$

where m_i = frequency (count) of item i in the stream.

2) Why many variables?

- One random sample can give a very noisy estimate.
- Therefore, AMS maintains many independent variables:

$$X_1, X_2, X_3, \cdots, X_t$$

- But the number of variables is limited (small), so memory usage is small.

Instead of storing one counter for every distinct item, we store only a limited number of sampled counters.

↓

This saves memory.

1) Pick and keep track of many variables X:

- AMS does not store the frequency of every distinct item. Instead, it keeps a small number of sampled variables X.
- For each variable X we store two things:

X.el	X.el corresponds to the item i that this variable is currently associated with.
X.val	X.val corresponds to the count (frequency) of that item i (relevant count).

So, a variable is a pair: $X = (X.el, X.val)$

Example of a variable X:

Suppose the stream is: a, b, a, c, a, b, d, a, ...

Assume one variable has sampled item 'a' at some position.

As we see more 'a's in the stream, X.val increases.

So after processing some part of the stream,

$X = (a, 3)$ means we are tracking item 'a' and its (relevant) count is 3.

Note: Since each X needs a count in main memory, the total number of X's that we can keep is limited.

Our Goal:

To compute (estimate) the 2nd moment / Surprise Number

$$S = \sum_i m_i^2$$

using only a small number of sampled variables.

One Random Variable (X)

- **How to set *X.val* and *X.el*?**
  - Assume stream has length ***n*** (we relax this later)
  - Pick some random time ***t*** (***t<n***) to start, so that any time is equally likely
  - Let at time ***t*** the stream have item ***i***. ***We set X.el = i***
  - Then we maintain count ***c*** ***(X.val = c***) of the number of ***i***s in the stream starting from the chosen time ***t***
- **Then the estimate of the 2nd moment ($\sum_i m_i^2$) is:**

$$S = f(X) = n(2 \cdot c - 1)$$

  - Note, we will keep track of multiple **X**s, ($\mathbf{X_1}$, $\mathbf{X_2}$,… $\mathbf{X_k}$) and our final estimate will be $S = 1/k \sum_j^k f(X_j)$

One Random Variable (X) in AMS Method

Goal: We want to estimate the 2nd moment (Surprise Number)

$$F_2 = \sum_i m_i^2$$ where m_i = total frequency of item i in the entire stream.

① Assume stream has length n.

Let the stream be: A, B, A, C, A, B, A, D, A, B

Positions: 1 2 3 4 5 6 7 8 9 10

So, $n = 10$.

② Pick a random time t (t < n).

- Every position is equally likely.
- Suppose we randomly pick $t = 3$.

③ Set X.el.

- At time $t = 3$, the stream item is 'A'.
- So, $X.el = A$ (the element selected).

④ Compute X.val = c.

- From time $t = 3$ to the end, count how many times item 'A' appears.
- Stream from position 3 onwards:

  , (A) , C , (A) , B , (A) , D , (A) , B

  There are 4 A's.

  So, $X.val = c = 4$ (suffix count of A from time t).

⑤ Estimate given by this X.

AMS defines the estimate from one variable X as

$$f(X) = n(2c - 1)$$

Here, $n = 10$, $c = 4$

So, $f(X) = 10(2 \times 4 - 1)$

$= 10(8 - 1)$

$= 10 \times 7 = 70$ (one random estimate)

(For understanding)

- Actual frequencies in the whole stream:

  $m_A = 5$, $m_B = 3$, $m_C = 1$, $m_D = 1$

- Actual 2nd moment:

  $F_2 = 5^2 + 3^2 + 1^2 + 1^2$

  $= 25 + 9 + 1 + 1 = 36$ (true value)

Why formula n(2c - 1)?

- If an item occurs m times in total, the possible values of c (depending on where we sample it) are 1, 2, ..., m.
- Average of (2c - 1) over all m choices equals m.
- So, $E[f(X)] = E[n(2c - 1)] = n \times m$ for that item.
- Adding over all items gives $\sum_i m_i^2$. (Proof in theory.)

⑥ We use many such variables.

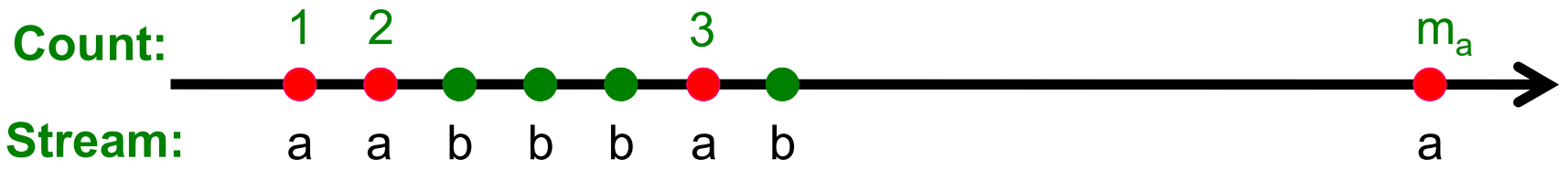
- We keep k independent variables: $X_1, X_2, \ldots, X_k$.
- For each X_j, compute $f(X_j) = n(2c_j - 1)$.
- Final estimate of F_2 (Surprise Number) is the average:

$$\hat{F}_2 = \frac{1}{k} \sum_{j=1}^{k} f(X_j)$$ (Averaging reduces randomness.)

Summary of one random variable X

1. Pick random position t $(1 \le t \le n)$.
2. Let X.el be the item at position t.
3. Let X.val = c = count of that item from time t till the end.
4. Estimate contributed by X is $f(X) = n(2c - 1)$.

Expectation Analysis



- **2nd moment is $S = \sum_i m_i^2$**
- c_t … number of times item at time t appears from time **t** onwards ($c_1=m_a$, $c_2=m_a-1$, $c_3=m_b$)
- $E[f(X)] = \frac{1}{n}\sum_{t=1}^{n} n(2c_t - 1)$

  $= \frac{1}{n}\sum_i n\,(1 + 3 + 5 + \cdots + 2m_i - 1)$

m_i … total count of item i in the stream (we are assuming stream has length **n**)

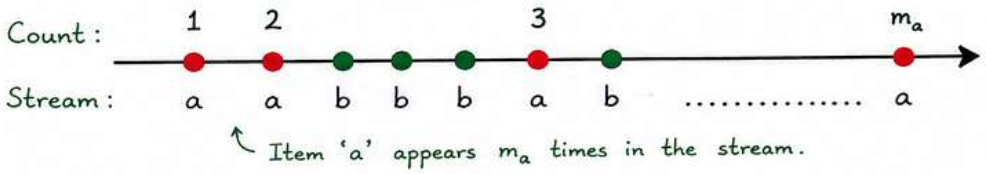
Group times by the value seen

Time t when the last i is seen ($c_t=1$)

Time t when the penultimate i is seen ($c_t=2$)

Time t when the first i is seen ($c_t=m_i$)

Expectation Analysis (Why AMS estimator is Unbiased)



- 2nd moment (Surprise Number) is

$$S = F_2 = \sum_i m_i^2$$

- Let c_t = number of times the item at time t appears from time t onwards (including time t).
- We randomly pick one time t ($1 \le t \le n$, where n = stream length).
- For the chosen time t, the AMS estimator is

$$f(X) = n(2c_t - 1)$$

Key Identity (Sum of first m odd numbers)

$$1 + 3 + 5 + \cdots + (2m-1) = m^2 \quad \text{(very useful!)}$$

Proof (by induction or known result). For $m = 1$, LHS $= 1 = 1^2$.
Assume true for $m-1$. Then,

$(1 + 3 + \cdots + (2m-3) + (2m-1) = (m-1)^2 + (2m-1)$
$= m^2 - 2m + 1 + 2m - 1 = m^2$. Hence true.

Expectation of f(X)

Since every time t ($1 \le t \le n$,) is equally likely (probability $\frac{1}{n}$),

$$E[f(X)] = \frac{1}{n}\sum_{t=1}^{n} f(X_t) = \frac{1}{n}\sum_{t=1}^{n} n(2c_t - 1) \quad \text{(by defn)}$$

$$= \sum_{t=1}^{n} (2c_t - 1) \quad (n \text{ cancels out})$$

What are possible values of c_t for item 'a'?

Suppose 'a' occurs m_a times in the stream.
Look at the m_a positions where 'a' occurs.

Occurrence of 'a'	1st a	2nd a	3rd a	...	$m_a{}^{th}$ a
Suffix count c_t	m_a	$m_a - 1$	$m_a - 2$	...	1

Why? Because:

- At the first occurrence, from there to the end, we will see m_a times.
- At the second occurrence, we will see $m_a - 1$ times.
- ...
- At the last occurrence, we will see 1 time.

Hence, c_t takes all values: $m_a, m_a - 1, \cdots, 2, 1$

Contribution of item 'a' to the expectation

For item 'a', the possible values of $(2c_t - 1)$ are:

$$(2m_a - 1), (2m_a - 3), (2m_a - 5), \cdots, 3, 1 \quad \text{(first } m_a \text{ odd numbers)}$$

Sum of these values $= 1 + 3 + 5 + \cdots + (2m_a - 1) = m_a^2$

So, the total contribution of item 'a' over all m_a possible times of sampling is exactly m_a^2.

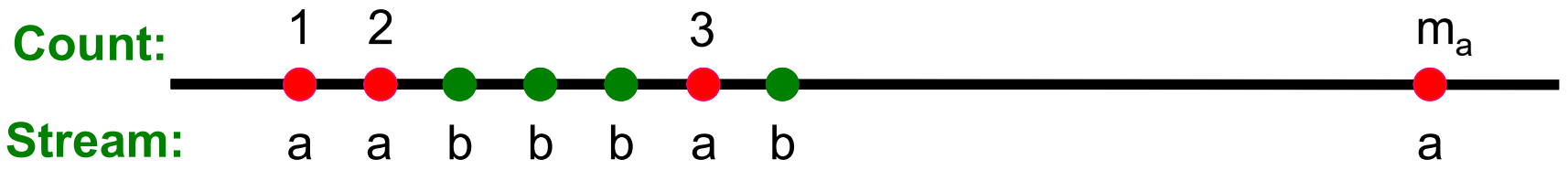
Now, group the times t by item. For each item i that occurs m_i times, its contribution to the above sum is m_i^2 (by the argument above).
Therefore,

$$E[f(X)] = \sum_i m_i^2 = F_2 = S \quad \text{(Unbiased)}$$

Summary:

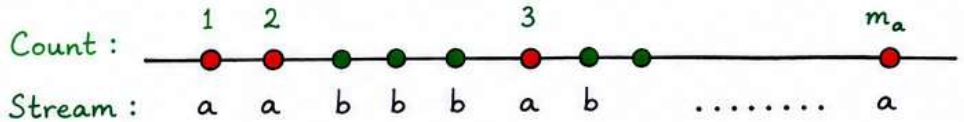
- Pick a random time t.
- Let the item at time t be i.
- Compute c_t = suffix count of i from t onwards.
- Estimator from this sample: $f(X) = n(2c_t - 1)$.
- Average over k independent samples gives $\hat{F}_2 = \frac{1}{k}\sum_{j=1}^{k} f(X_j)$.

Expectation Analysis



- $E[f(X)] = \frac{1}{n}\sum_i n\,(1+3+5+\cdots+2m_i-1)$
  - Little side calculation: $(1+3+5+\cdots+2m_i-1) = \sum_{i=1}^{m_i}(2i-1) = 2\frac{m_i(m_i+1)}{2} - m_i = (m_i\ )^2$
- **Then $\boldsymbol{E[f(X)] = \frac{1}{n}\sum_i n\,(m_i)^2}$**
- **So, $\boldsymbol{E[f(X)] = \sum_i (m_i)^2 = S}$**
- **We have the second moment (in expectation)!**

Expectation Analysis (Final Step)



- Recall, for one random variable X,

$$f(X) = n(2c_t - 1)$$ (where c_t = suffix count of the item at time t)

- We already had :

$$E[f(X)] = \frac{1}{n}\sum_{t=1}^{n} n\,(2c_t - 1)$$

$$= \frac{1}{n}\sum_{i} n \cdot (1 + 3 + 5 + \cdots + 2m_i - 1)$$

[Grouping the n time positions by the item i seen at that time]

- Little side calculation :

For an item i that occurs m_i times in the stream, the possible values of c_t are : $\underbrace{m_i, m_i - 1, m_i - 2, \ldots, 2, 1}_{(m_i \text{ values})}$

Therefore,

$$1 + 3 + 5 + \cdots + 2m_i - 1 = \sum_{j=1}^{m_i} (2j - 1)$$

$$= 2\sum_{j=1}^{m_i} j - \sum_{j=1}^{m_i} 1$$

$$= 2 \cdot \left(\frac{m_i(m_i+1)}{2}\right) - m_i$$

$$= m_i(m_i+1) - m_i$$

$$= m_i^2 + m_i - m_i$$

$$= m_i^2$$

$$\left[\sum_{j=1}^{m_i} j = \frac{m_i(m_i+1)}{2}\right]$$

$$\sum_{j=1}^{m_i} (2j - 1) = 1 + 3 + 5 + \cdots + (2m_i - 1) = m_i^2$$

(Sum of first m_i odd numbers = m_i^2)

- Use this result in $E[f(X)]$:

$$E[f(X)] = \frac{1}{n}\sum_{i} n \cdot (m_i^2)$$

$$= \sum_{i} (m_i^2)$$ (n cancels out)

But the 2nd moment (Surprise Number) is

$$S = F_2 = \sum_{i} m_i^2$$

So, $E[f(X)] = \sum_{i} (m_i)^2 = S$ (Unbiased)

What does this mean?

- For one random sample, $f(X)$ may be greater or smaller than the true value S.
- But in expectation (average over many random samples), we get exactly the true second moment S.

Key Takeaway :

The AMS estimator $f(X) = n(2c_t - 1)$ is an unbiased estimator of the second moment F_2.

That is, $E[f(X)] = F_2 = S$

Higher-Order Moments

- **For estimating kth moment we essentially use the same algorithm but change the estimate:**
  - For **k=2** we used ***n* (2·c – 1)**
  - For **k=3** we use: ***n* (3·c^2 – 3c + 1)** (where **c=X.val**)
- **Why?**
  - **For k=2:** Remember we had $(1 + 3 + 5 + \cdots + 2m_i - 1)$ and we showed terms ***2c-1*** (for **c=1,…,m**) sum to ***m^2***
    - $\sum_{c=1}^{m} 2c - 1 = \sum_{c=1}^{m} c^2 - \sum_{c=1}^{m} (c-1)^2 = m^2$
    - **So: $2c - 1 = c^2 - (c-1)^2$**
  - **For k=3: c^3 - (c-1)^3 = 3c^2 - 3c + 1**
- **Generally:** Estimate $= n\,(c^k - (c-1)^k)$

HIGHER – ORDER MOMENTS (Using AMS Method)

★ **Goal:** Estimate the kth frequency moment: $\boxed{F_k = \sum_i m_i^k}$ where m_i = total count of item i in the stream.

★ **Idea (Same as before):**

- Pick a random time t $(1 \le t \le n)$, n = stream length.
- Let the item at time t be i. $(X.el = i)$
- Let $c = X.val$ = number of times item i appears from time t onwards (including time t).
- We keep k such independent variables $X_1, X_2, \ldots, X_k$
- Only the estimator (formula) changes with k.

Notation

t : chosen random position

c : suffix count of the item at time t.

n : total length of stream

★ **1) Case $k = 2$ (Second moment)**

We already used estimator: $\boxed{f(X) = n(2c-1)}$

But notice,

$$c^2 - (c-1)^2 = c^2 - (c^2 - 2c + 1) = 2c - 1$$

So, another way to write it is: $f(X) = n\left[c^2 - (c-1)^2\right]$

Why does this give m_i^2? (Telescoping sum)

If item i occurs m_i times, then possible values of c are:

$\boxed{1, 2, 3, \ldots, m_i}$ (depending on which occurrence we pick).

Sum over all these occurrences:

$$\sum_{c=1}^{m_i}\left[c^2 - (c-1)^2\right] = (1^2 - 0^2) + (2^2 - 1^2) + \ldots + (m_i^2 - (m_i-1)^2)$$

$$= 1 + (4-1) + (9-4) + \ldots + (m_i^2 - (m_i-1)^2)$$

$$= \underline{\underline{m_i^2}}$$

Hence, contribution of item i to expectation is m_i^2.

★ 2) Case k = 3 (Third moment)

Estimator : $\boxed{f(X) = n\left[c^3 - (c-1)^3\right]}$

Expand :

$$(c-1)^3 = c^3 - 3c^2 + 3c - 1$$

So, $c^3 - (c-1)^3 = c^3 - (c^3 - 3c^2 + 3c - 1)$

$$= 3c^2 - 3c + 1$$

$$\Rightarrow \boxed{f(X) = n\,(3c^2 - 3c + 1)}$$

Again, for item i occurring m_i times :

$$\sum_{c=1}^{m_i}\left[c^3 - (c-1)^3\right] = m_i^{\,3}$$

(Telescoping)

★ 3) General Case (k^{th} moment)

Estimator : $\boxed{f(X) = n\left[c^k - (c-1)^k\right]}$

For item i occurring m_i times :

$$\sum_{c=1}^{m_i}\left[c^k - (c-1)^k\right] = m_i^{\,k}$$

(Telescoping)

★ 4) Unbiasedness

Each time position t is chosen with probability $\frac{1}{n}$, hence

$$E\left[f(X)\right] = \sum_i m_i^{\,k} = F_k = S_k \,.$$ (Unbiased estimator)

So, by using many independent samples and averaging, we can estimate any higher-order moment.

Combining Samples

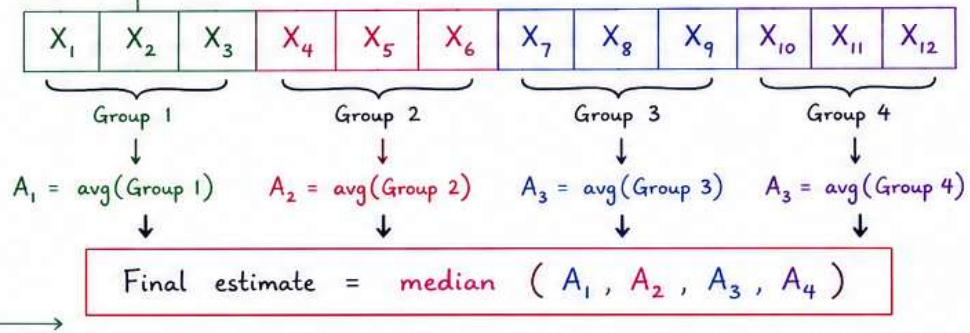
- **In practice:**
  - Compute $f(X) = n(2c - 1)$ for as many variables X as you can fit in memory
  - Average them in groups
  - Take median of averages
- **Problem: Streams never end**
  - We assumed there was a number n, the number of positions in the stream
  - But real streams go on forever, so n is a variable – the number of inputs seen so far

COMBINING SAMPLES (AMS in Practice and Open-Ended Streams)

1) In practice :

- One random variable X gives estimate : $f(X) = n(2c-1)$
- But one sample can be noisy (may be too high or too low) so we generate many independent variables.
- Steps :
  1. Compute $f(X) = n(2c-1)$ for as many variables X as you can fit in memory.
  2. Average them in groups.
  3. Take median of those group averages as final estimate.

Example : Suppose we can keep 12 samples.



Why median of averages?
Averages reduce variance. Taking median protects the result from a bad group (outlier). This is called "median-of-means".

Summary

- Use many AMS variables (as many as memory allows).
- Group them → average each group → take median of group averages.
- Deal with infinite streams by maintaining uniform random sample from all positions seen so far (using reservoir sampling).

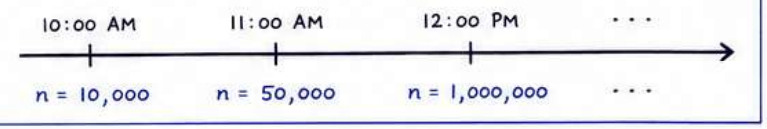
2) Problem : Streams never end

- In the analysis earlier, we assumed the stream has length n, i.e., total number of positions in the stream is n.

> n = number of positions in the stream (total length)

- But real data streams go on forever (e.g., search queries, network packets, sensor readings, clicks, ...).
- Therefore n is not fixed. It keeps increasing.

At different times :



- So we can only say :

> n = number of inputs seen so far (current stream length)

- Why is this a problem?
  AMS needs us to pick a random position t uniformly from $\{1, 2, \ldots, n\}$, but if n is not known in advance, how do we pick such a position?

Solution : We need a way to maintain a random sample from all positions seen so far, even though the stream may never end. This is what "Reservoir Sampling" helps us to do.

Streams Never End: Fixups

- **(1)** The variables ***X*** have ***n*** as a factor – keep ***n*** separately; just hold the count in ***X***
- **(2)** Suppose we can only store ***k*** counts. We must throw some ***X***s out as time goes on:
  - **Objective:** Each starting time ***t*** is selected with probability ***k/n***
  - **Solution: (fixed-size sampling!)**
    - Choose the first ***k*** times for ***k*** variables
    - When the nth element arrives (***n*** > ***k***), choose it with probability ***k/n***
    - If you choose it, throw one of the previously stored variables **X** out, with equal probability

STREAMS NEVER END : FIXUPS

(1) First Fix : Don't store n inside every variable

- Each AMS variable X gives estimate $f(X) = n(2c-1)$
- Earlier it looked like every X must store n.
- But n = number of elements seen so far is the same for all X's.
- So keep one global counter n.

> Each X only keeps the sampled element and its suffix count c.

(2) Second Fix : We can store only k counts (k variables)

But the stream goes on forever.
We must decide which starting times (positions) to keep.

> **Objective :** After seeing n elements, every starting time t ($1 \le t \le n$) should be selected with probability $\frac{k}{n}$.

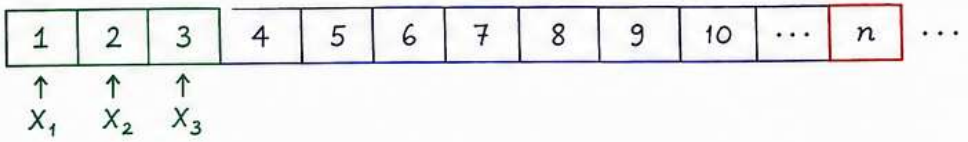
Solution : Fixed-size Sampling (Reservoir Sampling)

① Choose the first k times for k variables.
② When the nth element arrives ($n > k$), choose it with probability $\frac{k}{n}$.
③ If you choose it, throw (replace) one of the previously stored variables X out, with equal probability $\frac{1}{k}$ each.

> **Why $\frac{k}{n}$?** After n elements, we want every position (1 to n) to have the same chance to be in the reservoir.
> Since we keep only k positions, the chance for any given position should be exactly $\frac{k}{n}$.

How it works :

Suppose $k = 3$ (we can store 3 samples)

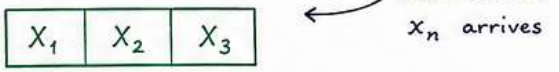


- Initially (first k elements) : store all.
- At time n ($n > k$) :
  - Accept the new element with probability $\frac{k}{n}$.
  - If accepted, randomly remove one of X_1, X_2, X_3 (each with probability $\frac{1}{k}$) and replace it with the new one.

> Thus, after seeing n elements, each of the n positions has probability $\frac{k}{n}$ of being in our reservoir (k samples).

Example :

Let $k = 3$.
When 10th element arrives, $P(\text{accept}) = \frac{3}{10}$.
When 100th element arrives, $P(\text{accept}) = \frac{3}{100}$.



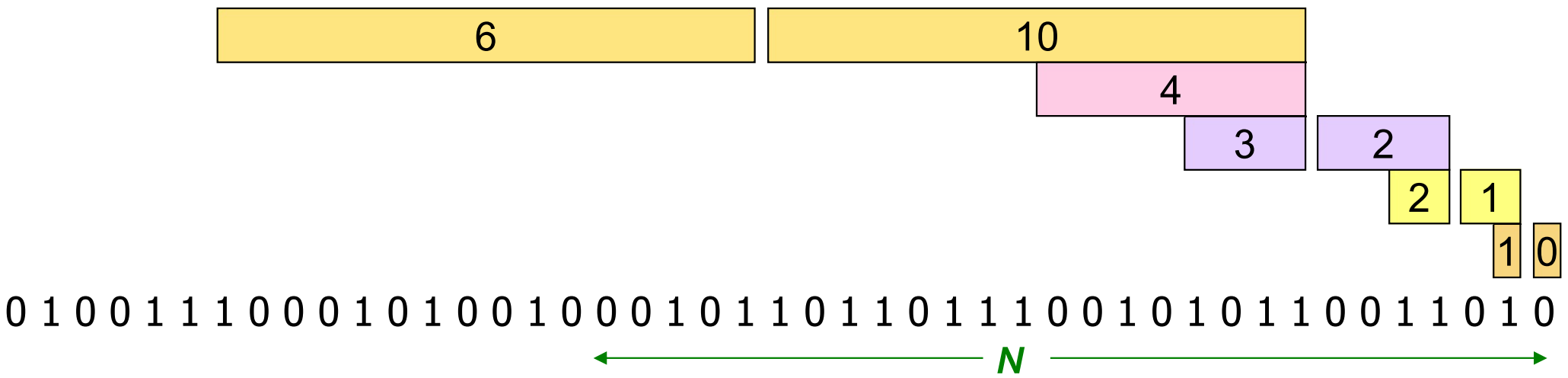
1) Select x_n with probability $\frac{k}{n}$
2) If selected → replace one of X_1, X_2, X_3 chosen uniformly at random (prob = $\frac{1}{k}$ each).

> **Key Takeaway :**
> - Maintain k samples using reservoir sampling.
> - Use one global n (items seen so far).
> - Each stored sample X keeps its suffix count c.
> - Compute estimate $f(X) = n(2c-1)$ for each X, then combine (group averages → median of averages).

Counting Itemsets

Counting Itemsets

- **New Problem:** **Given a stream, which items appear more than *s* times in the window?**
- **Possible solution:** Think of the stream of baskets as one binary stream per item
  - **1** = item present; **0** = not present
  - Use **DGIM** to estimate counts of **1**s for all items



COUNTING ITEMSETS (in a Sliding Window)

- **New Problem:** Given a data stream (of baskets/transactions), which items appear more than s times in the most recent N baskets (sliding window)?
- **Possible Solution:** Think of the stream of baskets as one binary stream per item.
  - 1 = item present in the basket
  - 0 = item not present in the basket
  - Use DGIM (Datar-Gionis-Indyk-Motwani) algorithm to estimate counts of 1s for all items.

Let the stream of baskets be:

time →	1	2	3	4	5	6	7	8	9	10	11	...
basket	B_1	B_2	B_3	B_4	B_5	B_6	B_7	B_8	B_9	B_{10}	B_{11}	...

Example of baskets:

$B_1 = \{A, B\}$ $B_2 = \{B, C\}$ $B_3 = \{A, C\}$ $B_4 = \{A, B, C\}$

$B_5 = \{B\}$ $B_6 = \{A\}$ $B_7 = \{C\}$ $B_8 = \{A, B\}$ $B_{10} = \{A, C\}$ ...

For each item we create a binary stream.

Item A :	1	0	1	1	0	1	0	1	0	1	...
Item B :	1	1	0	1	1	0	0	1	1	0	...
Item C :	0	1	1	1	0	0	1	0	1	1	...

Goal:

Given window size N and threshold s, find items i such that $count_i$ (in last N baskets) $> s$

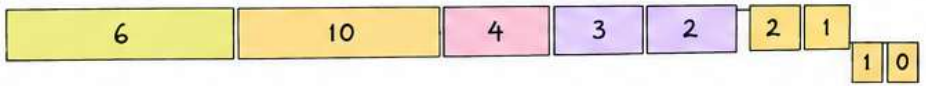
How it works:

1. For each item i, form its binary stream: 1 if item i appears in the basket, else 0.
2. Consider only the most recent N bits (sliding window).
3. Use DGIM algorithm to estimate the number of 1s in this window.
4. If estimated count $> s$, report the item as frequent in the current window.

What does DGIM store?

It keeps buckets of 1s in exponentially increasing sizes.
At most two buckets of the same size.
These buckets summarize the recent history and allow us to estimate the number of 1s in the last N positions.

Example of DGIM buckets for one item (window size N):



Below is the corresponding binary stream (most recent bits on the right side):

0 1 0 0 1 1 1 0 0 1 0 1 0 1 0 1 0 0 0 1 0 0 1 0 1 0 1 1 1 0 0 1 0 0 1 1 0 0 1 1 0

← N (window size) →

Interpretation:

- Each item has its own binary stream.
- DGIM summarizes that stream using few buckets.
- We can query: "How many 1s in the latest N bits?"
- If estimate $> s$ → item is frequent in current window.

This helps us find currently hot items (more than s appearances) in the most recent N transactions.

Extensions

- **In principle, you could count frequent pairs or even larger sets the same way**
  - **One stream per itemset**
- **Drawbacks:**
  - Only approximate
  - **Number of itemsets is way too big**

EXTENSIONS : COUNTING ITEMSETS (PAIRS AND LARGER SETS)

* In principle, we can extend the idea from single items to pairs and even larger itemsets :
  - One stream per itemset.

⇒ How does it work ?

- For any itemset S (for example, $S = \{A, B\}$), create a binary stream.

> 1 = the itemset S occurs in the basket
> 0 = otherwise

Example : Let the stream of baskets be (window size = N)

Time	1	2	3	4	5	6	7	8	9	10	...
Basket	{A,B,C}	{A,C}	{A,B}	{B,C}	{A,B,D}	{B}	{A,B,C}	{A,D}	{A,B}	{C}	...

⇒ For item A : 1 1 1 0 1 1 1 1 0 ...

⇒ For item B : 1 0 1 1 1 1 0 1 0 ...

⇒ For pair {A,B} : 1 0 1 0 1 1 0 1 0 ...

⇒ For triple {A,B,C} : 1 0 0 0 0 0 1 0 0 ...

Each of these binary streams can be given to DGIM to estimate the number of 1s in the most recent N positions.

> Hence, we can know approximately how many times an item / pair / triple / ... has appeared in the last N baskets.

Can we do this for all itemsets ?

Yes ! In theory, we can do the same for every possible itemset.

- One binary stream (and hence one DGIM structure) per itemset.

But ...

* Drawbacks :

1) Only approximate :
   DGIM itself gives approximate counts of 1s, so the itemset frequencies are approximate.

2) Number of itemsets is way too big :

> If there are d distinct items in the universe, number of non-empty itemsets = $2^d - 1$

Examples :

- For $d = 10$ items : $2^{10} - 1 = 1,023$ itemsets
- For $d = 20$ items : $2^{20} - 1 = 1,048,575$ itemsets
- For $d = 100$ items : $2^{100} - 1 \approx 1.27 \times 10^{30}$ itemsets !!!

> Hence, maintaining one DGIM per itemset is completely infeasible in practice.

Conclusion :

While the idea can be extended from single items to pairs and larger itemsets, the exponential number of itemsets makes the naive extension impractical.

Exponentially Decaying Windows

- **Exponentially decaying windows: A heuristic for selecting likely frequent item(sets)**
  - **What are "currently" most popular movies?**
    - Instead of computing the raw count in last ***N*** elements
    - Compute a **smooth aggregation** over the whole stream
- If stream is $a_1, a_2, \ldots$ and we are taking the sum of the stream, take the answer at time t to be: $= \sum_{i=1}^{t} a_i (1-c)^{t-i}$
  - **c** is a constant, presumably tiny, like 10^{-6} or 10^{-9}
- **When new a_{t+1} arrives:**
  Multiply current sum by **(1-c)** and add a_{t+1}

EXPONENTIALLY DECAYING WINDOWS (EDW)

* **Why do we need it?**

We want to answer : "What is popular right now?"

- Counting all past occurrences is misleading.
- Old data should gradually lose importance.
- Sliding window → hard cut off (keep last N only).
- EDW → smooth forgetting (no hard cut off).

* **Definition :**

Let the stream be $a_1, a_2, \ldots, a_t$

At current time t, define the exponentially decayed sum as :

$$S_t = \sum_{i=1}^{t} a_i \, (1-c)^{t-i}$$

where $0 < c < 1$

c is a small constant, e.g., 10^{-6} or 10^{-9}

Here, $(1-c)^{t-i}$ is the weight of element a_i.

→ For the most recent element a_t → weight = 1

→ For a_{t-1} → weight = $(1-c)$

→ For a_{t-2} → weight = $(1-c)^2$

...

Recent high weight

Old low weight

* **Example :**

Let $c = 0.1 \Rightarrow 1 - c = 0.9$

Suppose stream values : $a_1 = 1$, $a_2 = 0$, $a_3 = 1$

At time $t = 3$,

$$S_3 = a_1 (0.9)^2 + a_2 (0.9)^1 + a_3 (0.9)^0$$
$$= 1 (0.81) + 0 (0.9) + 1 (1)$$
$$= \underline{1.81}$$

Older value still contributes, but with smaller weight.

Efficient Update (Key Idea) :

We do NOT recompute the whole sum at every step.

If the current value is S_t, when new value a_{t+1} arrives :

$$S_{t+1} = (1-c) S_t + a_{t+1}$$

Very easy and very fast to maintain!

i.e., multiply current sum by $(1-c)$, then add the new value.

Example : Let $c = 0.1$ $(1-c = 0.9)$

Suppose current $S_t = 5$.

- New value $a_{t+1} = 1$

  $S_{t+1} = 0.9(5) + 1 = \underline{5.5}$

- Next value $a_{t+2} = 0$

  $S_{t+2} = 0.9(5.5) + 0 = \underline{4.95}$

If an item stops appearing, its score automatically decays over time.

* **Intuition :**

Time →	$t-4$	$t-3$	$t-2$	$t-1$	t
Weights →	$(1-c)^4$	$(1-c)^3$	$(1-c)^2$	$(1-c)$	1

New data gets full weight (1).

Old data fades / exponentially.

* **How used for frequent items / itemsets ?**

For each item or itemset :

$$a_t = \begin{cases} 1, & \text{if it appears in the basket at time } t \\ 0, & \text{otherwise} \end{cases}$$

Maintain $S_{t+1} = (1-c) S_t + a_{t+1}$

Large $S \Rightarrow$ item is hot / popular currently.

Summary : EDW gives more importance to recent events and gradually forgets the old ones. It answers "What's hot now?" in a smooth and memory-efficient way.

Example: Counting Items

- If each a_i is an "item" we can compute the **characteristic function** of each possible item x as an Exponentially Decaying Window
  - That is: $\sum_{i=1}^{t} \delta_i \cdot (1-c)^{t-i}$
    where δ_i=1 if a_i=x, and 0 otherwise
  - Imagine that for each item x we have a binary stream (1 if x appears, 0 if x does not appear)
  - New item x arrives:
    - Multiply all counts by **(1-c)**
    - Add **+1** to count for element x
- **Call this sum the "weight" of item x**

Example : Counting Items using Exponentially Decaying Windows (EDW)

* **Goal :** We want to know "what is popular right now?" Instead of counting all past occurrences equally, we give more importance to recent ones and gradually forget the old ones.

1) Each item → binary stream

Consider a stream of items (e.g., movie views):

A , B , A , C , A , B , B , A C , . . .

For any item x, create its own binary stream :

$$\delta_i = \begin{cases} 1, & \text{if } a_i = x \\ 0, & \text{otherwise} \end{cases}$$

Here a_i is the item at time i.

Example : For item A, the binary stream is

1	0	1	0	1	0	1	0	. . .
↑	↑	↑	↑	↑	↑	↑	↑	↑
a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9 ...

2) Exponentially decayed count (Weight) of item x

At current time t, the weight (decayed count) of item x is:

$$W_x(t) = \sum_{i=1}^{t} \delta_i \cdot (1-c)^{t-i}$$

$0 < c < 1$ (small constant, e.g., 10^{-6} or 10^{-9})

- $(1-c)^{t-i}$ is the weight of the occurrence at time i
- Recent events (i close to t) get high weight.
- Older events (i far from t) get tiny weight.

We call $W_x(t)$ the "weight" of item x.
Large weight → item has been frequent recently.

3) Efficient Update Rule

We do NOT recompute the sum every time.
Suppose current weight of item x is $W_x(t)$.
When next item a_{t+1} arrives :

$$W_x(t+1) = (1-c)\, W_x(t) + \delta_{t+1}$$

$(1-c)$ → decay old weight
$+ \delta_{t+1}$ → add 1 if new item is x

For every time step :
1. Multiply all weights by (1-c) (decay)
2. For the arriving item x, add +1 to its weight.

4) Simple Example

Let $c = 0.1 \Rightarrow 1-c = 0.9$

Suppose current weight of item A is $W_A = 5$.

Time	Arriving item	Update of W_A	New W_A
t	–	Current weight	5.00
t+1	Item is NOT A	$W_A \leftarrow 0.9 \times 5$	4.50
t+2	Item IS A	$W_A \leftarrow 0.9 \times 4.5 + 1$	5.05
t+3	Item NOT A	$W_A \leftarrow 0.9 \times 5.05$	4.545
t+4	Item IS A	$W_A \leftarrow 0.9 \times 4.545 + 1$	5.0905

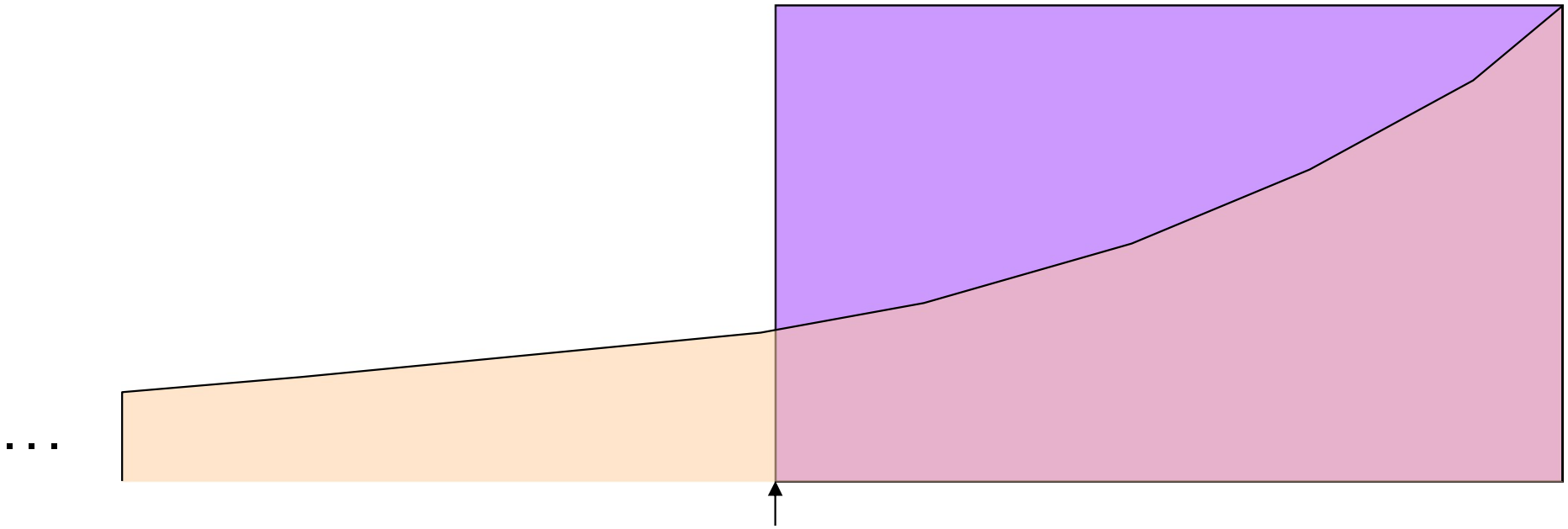
5) Intuition

- Each item has a score (weight).
- Old scores fade by factor (1-c) at every step.
- If item appears, we give it a boost of +1.
- Items that appear often recently will have large weights.

Summary

EDW gives a smooth, memory-efficient way to answer "What is currently popular?" for items in a stream.

Sliding Versus Decaying Windows

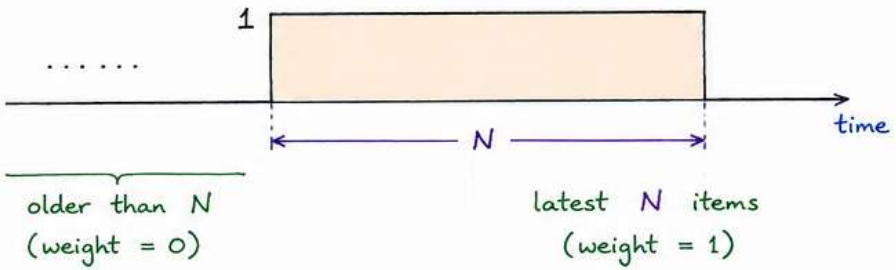


- **Important property:** Sum over all weights $\sum_t (1-c)^t$ **is 1/[1 – (1 – c)] = 1/c**

Sliding Versus Decaying Windows

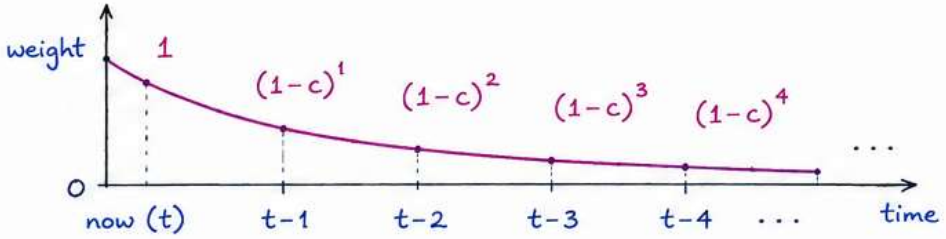
★ 1) Sliding Window

- We keep only the most recent N observations.
- Each of the latest N items has weight = 1.
- All items older than N have weight = 0.



2) Exponentially Decaying Window (EDW)

- No hard cut-off.
- Older items keep contributing, but their weights decay exponentially.

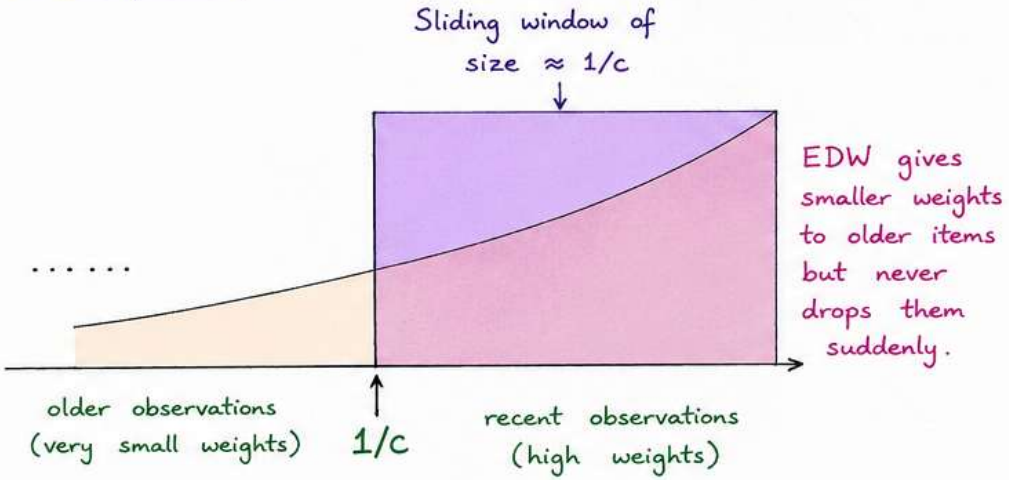


Weights of observations (from newest to older):

$1,\ (1-c),\ (1-c)^2,\ (1-c)^3,\ (1-c)^4,\ \ldots$

where $0 < c < 1$ (small constant)

3) Comparison (Picture from slide)



4) Important Property

The total weight of all observations in EDW is finite and equals $1/c$.

$$\text{Sum of all weights} = \sum_{t=0}^{\infty} (1-c)^t \quad \text{(geometric series)}$$

$$= \frac{1}{1-(1-c)} = \frac{1}{c} \quad \text{(since } |1-c|<1\text{)}$$

What does $1/c$ mean?

- EDW behaves roughly like a sliding window whose size is about $1/c$.

  Example:
  - If $c = 0.01 \Rightarrow 1/c = 100 \Rightarrow$ effective window ≈ 100
  - If $c = 0.001 \Rightarrow 1/c = 1000 \Rightarrow$ effective window ≈ 1000

Takeaway: Sliding window has a hard cut-off. Decaying window forgets smoothly and has an effective memory scale of about $1/c$.

Example: Counting Items

- **What are "currently" most popular movies?**
- **Suppose we want to find movies of weight > ½**
  - **Important property:** Sum over all weights $\sum_t (1-c)^t$ is 1/[1 – (1 – c)] = **1/c**
- **Thus:**
  - There cannot be more than **2/c** movies with weight of ½ or more
- So, **2/c** is a limit on the number of movies being counted at any time

Example : Counting Items (Movies)

★ 1) Our Goal

We want to answer the question:

What are "currently" most popular movies ?

2) Suppose we want movies of weight > 1/2

Using Exponentially Decaying Window (EDW), each movie x has a weight $W_x(t)$ at time t.

3) Important Property

From previous slide, we know that the total weight of all observations (over all movies) is:

$$\text{Total Weight} = \sum_{t=0}^{\infty} (1-c)^t = \frac{1}{1-(1-c)} = \frac{1}{c}$$

where $0 < c < 1$ is a small constant.

Meaning: The entire stream has only $1/c$ units of total weight to distribute among all movies.

Example: If $c = 0.01 \Rightarrow 1/c = 100$

Then maximum number of movies with weight $\geq 1/2$ is $2/c = 200$

Even if there are millions of movies, at most 200 can be "currently" popular (weight $\geq 1/2$) !

4) Why can there be at most $2/c$ movies with weight of $1/2$ or more ?

Suppose there are M movies each having weight at least $1/2$.

Then the total weight contributed by these M movies is at least:

$$M \times \left(\frac{1}{2}\right) = \frac{M}{2}$$

But total weight available in the whole stream is only $1/c$.

Therefore,

$$\frac{M}{2} \leq \frac{1}{c} \Rightarrow M \leq \boxed{\frac{2}{c}}$$

Thus: There cannot be more than $2/c$ movies with weight of $1/2$ or more.

5) What does this mean ?

- At any time, the number of "currently hot" movies (weight $\geq 1/2$) is limited to $2/c$.
- We can maintain counters only for such movies.
- Movies with weight $< 1/2$ can be ignored (they are not popular enough currently).

Key Takeaway: The property of EDW gives us a strong memory bound: At any time, we only need to keep track of at most $2/c$ movies.

Extension to Itemsets

- **Count (some) itemsets in an E.D.W.**
  - **What are currently "hot" itemsets?**
    - **Problem:** Too many itemsets to keep counts of all of them in memory
- **When a basket B comes in:**
  - Multiply all counts by **(1-c)**
  - For uncounted items in **B**, create new count
  - Add **1** to count of any item in **B** and to any **itemset** contained in **B** that is already being counted
  - **Drop counts < ½**
  - Initiate new counts (next slide)

Extension to Itemsets

★ **Goal:** Count (some) itemsets in an Exponentially Decaying Window (E.D.W.) and find which itemsets are currently "hot".

★ **Challenge:** Too many itemsets ($2^d - 1$ for d items). We cannot keep counts of all of them in memory.

Example basket: $B = \{A, B, C\}$ (arrives at time t)

Main Idea:
We maintain counts (weights) only for some promising itemsets. The weight of an itemset S at time t is denoted by $W(S, t)$.

④ **Drop counts $< 1/2$**

After decay (and possibly after step 3), if $W(S) < 1/2$, we delete S from memory.

We only keep itemsets whose weight is at least $1/2$. They are considered "currently hot enough".

⑤ **Initiate new counts (next slide)**

We may start tracking new larger itemsets that look promising based on the rule:

Start a count for itemset $S \subseteq B$ only if all proper subsets of S were already being counted BEFORE the arrival of this basket B.

What to do when a basket B comes in?

① **Decay all existing counts**

Every item / itemset that we are currently counting becomes a little less important.

For every tracked S: $W(S) \leftarrow (1-c) \cdot W(S)$ $\quad 0 < c < 1$ (small constant)

This represents the passage of time (older effects fade).

② **For uncounted items in B, create new count**

If an individual item in B is not being tracked yet, start tracking it with weight 0 (after decay) and then add 1 (see step 3).

In $B = \{A, B, C\}$, if C is new → create counter for C.

③ **Add 1 to count of any item in B and to any itemset contained in B that is already being counted**

For every tracked itemset S such that $S \subseteq B$ (S already exists), do:

$W(S) \leftarrow W(S) + 1$

This includes individual items (size-1 sets) and larger itemsets (pairs, triples, ...) that are already being tracked.

Example: In $B = \{A, B, C\}$

- Add 1 to A, B, C if already counted.
- Add 1 to $\{A,B\}, \{A,C\}, \{B,C\}$ if already counted.
- Add 1 to $\{A, B, C\}$ if already counted.

Summary of processing a basket B

1. Decay all counts by (1-c) → 2. Create counters for new items in B → 3. Add +1 to items in B and tracked itemsets contained in B → 4. Drop counts $< 1/2$ → 5. Initiate new counts for promising itemsets

EDW helps us focus on itemsets that are frequent recently while gradually forgetting old patterns.

Initiation of New Counts

- Start a count for an itemset $S \subseteq B$ if every proper subset of S had a count prior to arrival of basket B
  - **Intuitively:** If all subsets of **S** are being counted this means they are "**frequent/hot**" and thus S has a potential to be "**hot**"
- **Example:**
  - Start counting S={**i, j**} iff both **i** and **j** were counted prior to seeing B
  - Start counting S={**i, j, k**} iff {**i, j**}, {**i, k**}, and {**j, k**} were all counted prior to seeing B

Initiation of New Counts

★ **Problem:** In a stream of baskets, there are an enormous number of possible itemsets ($2^d - 1$ for d items). We cannot maintain a counter for every possible itemset.

★ **Idea:** Start a counter for an itemset S only when it has a good chance of becoming "hot" (frequent) in the future.

> ★ **Main Rule:**
>
> Start a count for an itemset $S \subseteq B$ (subset of the new arriving basket B) only if every proper subset of S had a count prior to arrival of basket B.

★ **Explanation / Intuition:**

If all subsets of S are already being counted, it means they are "frequent / hot". Therefore, S also has a potential to be "hot".

→ This is similar to the Apriori principle:

> If a set is frequent,
> all of its subsets must also be frequent.

★ **Example 1: Starting a pair**

- Suppose a new basket arrives: $B = \{i, j, k\}$
- Consider itemset $S = \{i, j\}$
- Proper subsets of S are: $\{i\}, \{j\}$
- If both $\{i\}$ and $\{j\}$ were already being counted before seeing basket B,

  $\Rightarrow$ Start counting $S = \{i, j\}$

★ **Example 2: Starting a triple**

- Consider itemset $S = \{i, j, k\}$ from basket $B = \{i, j, k\}$
- Proper subsets of S (all pairs) are: $\{i, j\}, \{i, k\}, \{j, k\}$
- If $\{i, j\}$, $\{i, k\}$, and $\{j, k\}$ were all being counted prior to seeing basket B,

  $\Rightarrow$ Start counting $S = \{i, j, k\}$

★ **Important Point:**

- The subsets must have been counted BEFORE the current basket B arrives.
- We should NOT use the current basket to create all subsets and then immediately start larger itemsets. This prevents explosion of counters.

Summary We build itemset counts gradually:
items → pairs → triples → larger itemsets ...

> Start tracking S only when all its proper subsets are already promising (being counted).

How many counts do we need?

- Counts for single items **< (2/c)·(avg. number of items in a basket)**
- **Counts for larger itemsets = ??**
- **But we are conservative about starting counts of large sets**
  - If we counted every set we saw, one basket of **20** items would initiate **1M** counts

How many counts do we need?

1) Counts for single items

From previous result:

At most $2/c$ items can have weight $\geq 1/2$ in an E.D.W.

But here each basket contains many items.

Let b = average number of items in a basket.

Each arriving basket contributes about b units of weight (instead of 1 unit in previous slide).

Total available weight for individual items $\approx b/c$

Since every retained item has weight at least $1/2$, the number of item counters needed is:

> Number of single-item counters $< \frac{2}{c} \times b$

(where b = avg. number of items in a basket)

So, counters for single items $< \frac{2}{c} \cdot$ (avg. number of items in a basket)

2) Counts for larger itemsets (pairs, triples, ...)

What about pairs, triples, larger sets?

Here we do NOT have such a simple small bound.

A basket with many items contains an enormous number of possible itemsets.

Example:

Suppose one basket contains 20 items.

Number of non-empty subsets (itemsets) $= 2^{20} - 1$

$= 1{,}048{,}575 \approx 1$ million

So, counting every possible set in one basket of 20 items would create about 1 million counters!

3) Why we are conservative about starting large sets

If we start counting every set that appears in every basket, memory will explode.

Example: One basket with 20 items → 1M possible sets.

Clearly impossible to maintain all these counters!

Therefore we are conservative about starting counts of large sets.

4) How we limit the number of large-set counters

We follow the rule from the previous slide:

Start counting an itemset S only if all proper subsets of S were already being counted BEFORE this basket arrives.

This way, counts grow gradually:

single items → promising pairs → promising triples → promising larger sets

So, instead of tracking all $2^d - 1$ possible sets for d items, we track only a small number of hot sets.

Key Takeaway:

- Single items: at most $< \frac{2}{c} \times$ (avg. number of items in a basket) counters.
- Larger itemsets: cannot be bounded so easily → hence we are conservative and start them only when their subsets are already promising (were being counted earlier).

Bloom Filter – Numericals

① Numerical 1 :- Construct and Query a Bloom Filter

A Bloom filter has 10 bits, initially all 0.

Two hash functions are used:

$h_1(x) = x \bmod 10$

$h_2(x) = (3x+1) \bmod 10$

Insert $S = \{12, 25, 37\}$.

Then test whether 25, 42, 17 may belong to S.

Solution:

Initial bit array ($m = 10$):

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

Insertion:

x	$h_1(x) = x \bmod 10$	$h_2(x) = (3x+1) \bmod 10$
12	2	7
25	5	6
37	7	2

Set positions 2, 5, 6, 7 to 1.

Bit array after insertion:

0	1	2	3	4	5	6	7	8	9
0	0	1	0	0	1	1	1	0	0

Query:

- **Query 25:**
  $h_1(25) = 5$, $h_2(25) = 6$
  $B[5] = 1$, $B[6] = 1$
  ⇒ Possibly present.
  (Actually present → True Positive)

- **Query 42:**
  $h_1(42) = 2$, $h_2(42) = 7$
  $B[2] = 1$, $B[7] = 1$
  ⇒ Possibly present.
  ($42 \notin S$ → False Positive)

- **Query 17:**
  $h_1(17) = 7$, $h_2(17) = 2$
  $B[7] = 1$, $B[2] = 1$
  ⇒ Possibly present.
  ($17 \notin S$ → False Positive)

Observation: Bloom filter never says "absent" for an item that is actually in the set (no false negatives), but may say "present" for an item not in the set (false positive).

---

② Numerical 2 :- False Positive Probability

A Bloom filter stores $m = 1000$ elements using a bit array of $n = 8000$ bits and $k = 4$ hash functions.

Estimate the false positive probability.

* Expected fraction of 1's in the bit array:

$1 - e^{-km/n} = 1 - e^{-0.5} \approx 0.3935$

= **39.35% bits are 1**

Solution:

Formula:

$$P_{FP} \approx (1 - e^{-km/n})^k$$

Now,

$$\frac{km}{n} = \frac{4 \times 1000}{8000} = 0.5$$

$$P_{FP} \approx (1 - e^{-0.5})^4$$

$$e^{-0.5} \approx 0.6065$$

$$1 - 0.6065 = 0.3935$$

$$P_{FP} \approx (0.3935)^4$$

$$\boxed{P_{FP} \approx 0.024 \ (\approx 2.4\%)}$$

---

③ Numerical 3 :- Find Optimal Number of Hash Functions

A Bloom filter stores $m = 2000$ items using $n = 16000$ bits.

Find the optimal number of hash functions k_{opt}.

Solution:

$$\boxed{k_{opt} = \frac{n}{m} \ln 2}$$

$$k_{opt} = \left(\frac{16000}{2000}\right) \ln 2$$

$$= 8 \times 0.693$$

$$\boxed{k_{opt} \approx 5.544} \Rightarrow \boxed{k \approx 6}$$

Hence, using 6 hash functions is optimal.

Flajolet – Martin (FM) Algorithm – Numericals

(1) Numerical 1 :- Basic FM Estimation

The hash values (in binary) of distinct stream elements are given below.

Let $r(a)$ be the number of trailing 0's.

Find $R = \max_a r(a)$ and estimate the number of distinct elements using $\hat{D} = 2^R$.

Item	Hash value (binary)	$r(a)$ (trailing zeros)
A	101100	2
B	011000	3
C	101010	1
D	110000	4
E	100100	2

(Actual distinct elements are 5 here, but FM is only an estimate.)

Solution :-

$r(A) = 2$, $r(B) = 3$, $r(C) = 1$,
$r(D) = 4$, $r(E) = 2$

$R = \max = 4$

Estimated distinct count

$\hat{D} = 2^R = 2^4 = 16$

---

(2) Numerical 2 :- FM with Repeated Items

Consider the stream (left to right):

A, B, C, A, D, B, E, A, C, F

Hash values (binary) of items are given below.

Find the FM estimate of the number of distinct elements.

Item	Hash value (binary)	$r(a)$
A	10100	2
B	11110	1
C	11000	3
D	10110	1
E	10000	4
F	11010	1

Actual distinct count = 6, but single FM estimate can be far off (high variance).

Solution :-

Only distinct items matter:

{A, B, C, D, E, F}

$R = \max r(a) = 4$

Estimated distinct count

$\hat{D} = 2^R = 2^4 = 16$

---

(3) Numerical 3 :- Multiple FM Estimates

Using 5 independent hash functions we obtain the following maximum number of trailing zeros (R_i).

i	1	2	3	4	5
R_i	9	10	10	11	18

(a) Find the FM estimates 2^{R_i} for each i.

(b) Compute the mean estimate.

(c) Compute the median estimate.

Note: Mean is badly affected by large outlier (2^{18}). Median (or geometric mean) is preferred.

Solution :-

(a) 2^{R_i} : 2^9 2^{10} 2^{10} 2^{11} 2^{18}

$= 512$ 1024 1024 2048 262144

(b) Mean $= \dfrac{512 + 1024 + 1024 + 2048 + 262144}{5}$

$= \dfrac{266752/5}{5} = 53350.4$

(c) Write estimates in sorted order:

512, 1024, 1024, 2048, 262144

Median = 1024

---

(4) Numerical 4 :- FM Error Bound (Heuristic)

Using m independent FM hash functions, the relative standard error of the estimate is approximately $\frac{1}{\sqrt{m}}$.

(a) If $m = 16$, what is the approx. RSE?

(b) To achieve RSE at most 10%, how many hash functions are needed?

Solution :-

(a) $\text{RSE} \approx \dfrac{1}{\sqrt{m}} = \dfrac{1}{\sqrt{16}} = \dfrac{1}{4} = 0.25 = 25\%$

(b) $\dfrac{1}{\sqrt{m}} \leq 0.10 \Rightarrow \sqrt{m} \geq 10 \Rightarrow m \geq 100$

At least 100 hash functions are needed.

Frequency Moments (F_k) – Numericals

① Numerical 1 :- Compute F_0, F_1, F_2, F_3

Stream (baskets/items):

A , B , A , C , A , B , D , A , B , C

Step 1: Compute frequency of each item.

Item	A	B	C	D
Frequency (m_i)	4	3	2	1

Step 2: Compute moments.

- F_0 (no. of distinct items) = 4
- $F_1 = \sum_i m_i = 4 + 3 + 2 + 1 = 10$
- $F_2 = \sum_i m_i^2 = 4^2 + 3^2 + 2^2 + 1^2 = 16 + 9 + 4 + 1 = 30$
- $F_3 = \sum_i m_i^3 = 4^3 + 3^3 + 2^3 + 1^3 = 64 + 27 + 8 + 1 = 100$

Answer: $F_0 = 4$, $F_1 = 10$, $F_2 = 30$, $F_3 = 100$

② Numerical 2 :- Same F_0 and F_1, Different F_2

Two different streams are given below.

Stream A (balanced)
Frequencies: 5 , 5 , 5 , 5
(4 distinct items)

Stream B (skewed)
Frequencies: 17 , 1 , 1 , 1
(4 distinct items)

Moment	Value
F_0	4
F_1	$5+5+5+5 = 20$
F_2	$5^2+5^2+5^2+5^2 = 25+25+25+25 = 100$

Moment	Value
F_0	4
F_1	$17+1+1+1 = 20$
F_2	$17^2+1^2+1^2+1^2 = 289+1+1+1 = 292$

Observation: F_0 and F_1 are same for both streams, but F_2 is very different. F_2 captures the "concentration" or "skewness".

③ Numerical 3 :- Higher Moments

Given item frequencies:

A:7 , B:4 , C:3 , D:2 , E:1

Item	A	B	C	D	E
Frequency (m_i)	7	4	3	2	1

- $F_0 = 5$
- $F_1 = 7+4+3+2+1 = 17$
- $F_2 = 7^2 + 4^2 + 3^2 + 2^2 + 1^2 = 49 + 16 + 9 + 4 + 1 = 79$
- $F_3 = 7^3 + 4^3 + 3^3 + 2^3 + 1^3 = 343 + 64 + 27 + 8 + 1 = 443$
- $F_4 = 7^4 + 4^4 + 3^4 + 2^4 + 1^4 = 2401 + 256 + 81 + 16 + 1 = 2755$

What do these moments mean?

- F_0 → diversity (how many different items)
- F_1 → total count (size of stream)
- F_2 → heavier weight on frequent items (sensitive to skew)
- Higher F_k give even more weight to very frequent items.

④ Numerical 4 :- Effect of Adding One More Occurrence

From Numerical 1 stream: A(4), B(3), C(2), D(1) → $F_2 = 30$

If one more 'A' arrives: A becomes 5.

New frequencies: A(5), B(3), C(2), D(1)

New $F_2 = 5^2 + 3^2 + 2^2 + 1^2 = 25 + 9 + 4 + 1 = 39$

A single extra occurrence to the most frequent item increases F_2 by 9.

AMS (Alon-Matias-Szegedy) Algorithm – Numericals

① Numerical 1 :- One AMS Sample

Stream (items) :

A , B , A , A , B , C , A , B

Total no. of items $n = 8$

$F_2 = \sum_i m_i^2$ (we'll compute later)

Suppose the algorithm randomly picks position $t = 3$.

X.el = item at position 3 = A

From position 3 onward :

A	A	B	C	A	B

X.val = no. of times A occurs from position 3 to the end (including position 3).

X.val = 3

Solution :

AMS estimate for this sample

$f(X) = n\,(2c - 1)$

$= 8\,(2\times3 - 1)$

$= 8\,(6 - 1)$

$= 8 \times 5$

$f(X) = 40$

Actual F_2 of the stream :

Frequencies : A : 4 B : 3 C : 1

$F_2 = 4^2 + 3^2 + 1^2$

$= 16 + 9 + 1$

$F_2 = 26$

One AMS sample (40) need not equal the actual F_2 (26).

② Numerical 2 :- Verify Unbiasedness of AMS

Use the same stream :

A , B , A , A , B , C , A , B ($n = 8$)

Let AMS choose t uniformly at random from $\{1, 2, 3, 4, 5, 6, 7, 8\}$.

Compute $f(X)$ for every possible t.

t (position)	1	2	3	4	5	6	8
Item at t	A	B	A	A	B	C	B
Occurrences of that item from t to end (c)	4	3	3	2	2	1	1
$f(X) = 8(2c-1)$	56	40	40	24	24	8	8

Since each t is equally likely,

$E[f(X)]$ = average of all $f(X)$'s

$= (56 + 40 + 40 + 24 + 24 + 8 + 8 + 8) / 8$

$= 208 / 8$

$E[f(X)] = 26$

Actual $F_2 = 26$

So, $E[f(X)] = F_2$

⇒ AMS is an unbiased estimator of F_2.

③ Numerical 3 :- Another Stream Example

Stream : A , A , A , B , B , C , C , C , C , D ($n = 10$)

Frequencies : A : 3 B : 2 C : 4 D : 1

So, actual $F_2 = 3^2 + 2^2 + 4^2 + 1^2$

$= 9 + 4 + 16 + 1 =$ **30**

Suppose AMS randomly picks position $t = 6$.

Item at position 6 is C.

1	2	3	4	5	6	7	8	9	10
A	A	A	B	B	C	C	C	C	D

From position 6 to end : C , C , C , C , D

C occurs $c = 4$ times.

$f(X) = n\,(2c - 1)$

$= 10\,(2\times4 - 1)$

$= 10\,(8 - 1)$

$= 10 \times 7$

$f(X) = 70$

Actual $F_2 = 30$, but this sample estimate is 70.

④ Numerical 4 :- Multiple AMS Samples

For the same stream as in Numerical 1, suppose we take 5 independent AMS samples and get :

Sample i	1	2	3	4	5
t (position)	1	3	6	7	8
Item	A	A	C	A	B
c (from t to end)	4	3	1	1	1
$f(X) = 8(2c-1)$	56	40	8	8	8

Mean estimate $= \frac{56 + 40 + 8 + 8 + 8}{5}$

$= 120 / 5$

$=$ **24**

Actual $F_2 = 26$. With more samples, the average gets closer to the true F_2.

Key Point : One sample may be high or low, but the average over many independent samples converges to the true F_2.